Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mmeas functions

Synopsis

The functions prefixed with Mmeas make up the Measurement module. The Measurement module allows you to find markers (for example, edges) in an image, based on differences in pixel intensities. Upon finding a marker, the module returns the marker's spatial reference position and measures features such as its width and angle. You can calculate measurements between two markers, such as the equation of the line that joins them. You can also use the Measurement module to specify explicit points, which you can use as a reference position. The Measurement module can operate on 8-bit or 16-bit unsigned grayscale images. Measurements are made and results are returned with subpixel accuracy.

Functions

MmeasAllocContext
MmeasAllocMarker
MmeasAllocResult
MmeasCalculate
MmeasControl
MmeasDraw
MmeasFindMarker
MmeasFree
MmeasGetResult
MmeasGetResultSingle
MmeasGetScore
MmeasInquire
MmeasRestoreMarker
MmeasSaveMarker
MmeasSetMarker
MmeasSetScore
MmeasStream

MmeasAllocContext

Synopsis

Allocate a measurement context.

Syntax

```
AIL_ID MmeasAllocContext(
    AIL_ID SysId,          //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ContextIdPtr   //out
)
```

Description

This function allocates a measurement context on the specified system. Aurora Imaging Library uses a measurement context to control the behavior of measurement operations (MmeasFindMarker() and MmeasCalculate()). You can modify the settings of a measurement context with MmeasControl().

Aurora Imaging Library allows you to specify **M_DEFAULT** as the identifier of the measurement context, when using MmeasFindMarker(), MmeasCalculate(), and MmeasInquire(); therefore, you need not call **MmeasAllocContext**(). However, you cannot specify **M_DEFAULT** as the identifier of the measurement context if want to modify it using MmeasControl(); to modify a measurement context, you must actually allocate it using **MmeasAllocContext**().

When the measurement context is no longer required, release it using MmeasFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the measurement context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which the to allocate the measurement context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextIdPtr

Specifies the address of the variable in which to write the measurement context identifier or specifies the data type that the function should use to return the measurement context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MEAS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the measurement context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated measurement context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the measurement context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MEAS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasAllocMarker

Synopsis

Allocate a measurement marker buffer.

Syntax

```
AIL_ID MmeasAllocMarker(
    AIL_ID SysId,          //in
    AIL_INT64 MarkerType,  //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *MarkerIdPtr    //out
)
```

Description

This function allocates a measurement marker buffer on the specified system. You can allocate an edge, stripe, circle, or point marker buffer.

Once allocated, you can specify a marker's essential characteristics using MmeasSetMarker(); for an edge, stripe, or circle marker, you can also specify score characteristics using MmeasSetScore(). For an edge, stripe, or circle marker, Aurora Imaging Library uses both essential and score characteristics to find the marker in a target image when you call MmeasFindMarker(). You cannot search for a point marker; you must define it at a specific location with MmeasSetMarker(). A point marker is typically used as a reference position when taking measurements between two markers using MmeasCalculate().

By default, when you allocate a marker buffer, it defines a single-occurrence marker. However, using MmeasSetMarker() with M_NUMBER, you can define an edge, stripe, or point marker as a multiple-occurrence marker. For edge and stripe markers, this allows you to search for multiple instances of the same image characteristics. For point markers, this allows you to define equidistant reference positions. You cannot define a multiple-occurrence circle marker.

When the measurement marker buffer is no longer required, release it using MmeasFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the measurement marker buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the marker buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

MarkerType INQ

Specifies the type of marker buffer to allocate.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of marker buffer to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CIRCLE	Specifies a circle marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE	Specifies an edge marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT	Specifies a point marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE	Specifies a stripe marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

MarkerIdPtr

Specifies the address of the variable in which to write the measurement marker buffer identifier or specifies the data type that the function should use to return the measurement marker buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement marker buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement marker buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MEAS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the measurement marker buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated measurement marker buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the measurement marker buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MEAS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasAllocResult

Synopsis

Allocate a measurement result buffer.

Syntax

```
AIL_ID MmeasAllocResult(
    AIL_ID SysId,           //in
    AIL_INT64 ResultType,   //in
    AIL_ID *MeasResultIdPtr //out
)
```

Description

This function allocates a measurement result buffer, on the specified system, to store results obtained from an MmeasCalculate() operation. Note that you do not need a result buffer for an MmeasFindMarker() operation; results are stored with the marker.

When the measurement result buffer is no longer required, release it using MmeasFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the measurement result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType INQ

Specifies the type of result buffer to allocate.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result buffer to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CALCULATE.	c		e	f	g	a		i		j	k	l	m	n	b

M_CALCULATE	Allocates a result buffer for an MmeasCalculate() operation.	c		e	f	g	a		i		j	k	l	m	n	b

MeasResultIdPtr

Specifies the address of the variable in which to write the measurement result buffer identifier or specifies the data type that the function should use to return the measurement result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MEAS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the measurement result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated measurement result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the measurement result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MEAS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasCalculate

Synopsis

Calculate measurements between two markers.

Syntax

```
void MmeasCalculate(
    AIL_ID ContextId,         //in
    AIL_ID Marker1Id,         //in
    AIL_ID Marker2Id,         //in
    AIL_ID MeasResultId,      //out
    AIL_INT64 MeasurementList //in
)
```

Description

This function calculates the specified measurements between the two specified markers. Before using an edge, stripe, or circle marker with this function, it must have been previously found in a target image (MmeasFindMarker()). Before using a point marker, its position must have been previously set (MmeasSetMarker()).

The settings of the specified measurement context control the behavior of this function; to adjust these settings, use MmeasControl(). Aurora Imaging Library stores the results of an **MmeasCalculate**() operation in the specified measurement result buffer; to obtain the results, use MmeasGetResult() or MmeasGetResultSingle(). If the markers were found in a calibrated image, you can specify the output units using MmeasControl() with M_RESULT_OUTPUT_UNITS. Aurora Imaging Library calculates the specified measurements according to the line joining the reference position of each marker (Marker1Id and Marker2Id).

For edge, stripe, and circle markers, the default reference position corresponds to the marker's found position. For an edge marker, the default reference position is the X- and Y-coordinates of the edge's maximum edgevalue. For a stripe marker, the default reference position is the center of the center line that is fit between the stripe's two outer edges. For a circle marker, the default reference position is the X- and Y-coordinates of the circle's center. For a point marker, the default reference position is the same as the position specified using MmeasSetMarker() with M_POSITION.

To change the reference position of any marker, use MmeasSetMarker() with M_MARKER_REFERENCE. Modifying the reference position only affects calculations between two markers (**MmeasCalculate**()); it does not, for example, affect how Aurora Imaging Library locates or returns the position of markers (MmeasFindMarker()).

If you specify multiple-occurrence markers, Aurora Imaging Library performs its calculations using the occurrences of the first marker and the corresponding occurrences of the second marker. Aurora Imaging Library limits the number of calculations to the smallest number of occurrences held in either marker. For example, if a marker contains one edge occurrence, Aurora Imaging Library only performs one calculation, regardless of the number of edge occurrences in the other marker.

Parameters

ContextId

Specifies the measurement context to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurement context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default measurement context of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b

Measurement context ID	Specifies a valid measurement context, allocated using MmeasAllocContext().	c		e	f	g	a		i		j	k	l	m	n	b

Marker1Id

Specifies the identifier of the first marker buffer to use for calculating measurements.

Marker2Id

Specifies the identifier of the second marker buffer to use for calculating measurements.

MeasResultId

Specifies the identifier of the result buffer in which to place results.

MeasurementList

Specifies which measurement(s) to calculate. This parameter must be set to one of the following values. To calculate more than one measurement, add the values together (for example, M_DISTANCE + M_ANGLE).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which measurement(s) to calculate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ANGLE + M_DISTANCE + M_LINE_EQUATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Calculates the angle of the line joining both markers, relative to the positive X-axis. The resulting angle can be any value between 0° and 360°.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE	Calculates the distance of the line joining both markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_EQUATION	Calculates the equation of the line joining both markers.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasControl

Synopsis

Control a measurement context or result buffer setting.

Syntax

```
void MmeasControl(
    AIL_ID ContextOrResultId, //out
    AIL_INT64 ControlType,    //in
    AIL_DOUBLE ControlValue   //in
)
```

Description

This function allows you to control a measurement context or result buffer setting. These settings control the behavior of measurement operations (MmeasFindMarker() and MmeasCalculate()), which subsequently affects the results that you obtain (MmeasGetResult() or MmeasGetResultSingle()) or draw (MmeasDraw()).

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MmeasControl()** internally calls the *AIL_DOUBLE* version of this function (MmeasControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MmeasControlInt64()).

Parameters

ContextOrResultId

Specifies the identifier of the measurement context or result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurement context or result buffer identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Measurement context ID	Specifies a valid measurement context, previously allocated using MmeasAllocContext().	c		e	f	g	a		i		j	k	l	m	n	b
Measurement result buffer ID	Specifies a valid measurement result buffer, previously allocated using MmeasAllocResult().	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For a measurement context
- For a measurement result buffer

The following ControlType and corresponding ControlValue parameter settings can be specified for a measurement context. In this case, you must set the ContextOrResultId parameter to a measurement context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement context																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_PIXEL_ASPECT_RATIO		Sets the pixel width to pixel height ratio. INQ																
	Value	Specifies the ratio. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO_INPUT		Sets whether to use M_PIXEL_ASPECT_RATIO to correct measurement inputs. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORRECTED	Specifies to correct measurement inputs with the specified aspect ratio. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMAL	Specifies to not correct measurement inputs (ratio of 1.0).	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO_OUTPUT		Sets whether to use M_PIXEL_ASPECT_RATIO to correct measurement results (output data). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORRECTED	Specifies to correct measurement results with the specified aspect ratio. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMAL	Specifies to not correct measurement results (ratio of 1.0).	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for a measurement result buffer. In this case, you must set the ContextOrResultId parameter to a measurement result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement result buffer																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	

M_RESULT_OUTPUT_UNITS	Sets whether to return results of MmeasCalculate() in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. Use MmeasSetMarker() with M_RESULT_OUTPUT_UNITS to control the units in which results found using MmeasFindMarker() are returned. INQ																
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MmeasGetResult() or MmeasGetResultSingle() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MmeasControlInt64 (AIL_ID ContextOrResultId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultId

See ContextOrResultId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MmeasControlDouble (AIL_ID ContextOrResultId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultId

See ContextOrResultId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasDraw

Synopsis

Draw expected marker characteristics or the characteristics of marker occurrences, or draw calculation results, in the destination image buffer or 2D graphics list.

Syntax

```
void MmeasDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID MarkerOrResultMeasId,   //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT Index,                 //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function performs the specified drawing operation on the expected marker characteristics (as set with MmeasSetMarker()) or the actual characteristics of the marker occurrences (as found with MmeasFindMarker()). This function can also perform the specified drawing operation on the result of a measurement calculation between two markers (MmeasCalculate()).

This function can execute multiple drawing operations simultaneously (for example, M_DRAW_SEARCH_REGION + M_DRAW_SEARCH_DIRECTION). Aurora Imaging Library performs the drawing operations in the specified destination image buffer or 2D graphics list.

You can draw settings and results, obtained relative to an offset, at the top-left corner of the destination image, using MgraControl() with M_DRAW_OFFSET_X and M_DRAW_OFFSET_Y, and zoom them using MgraControl() with M_DRAW_ZOOM_X and M_DRAW_ZOOM_Y. For more information, see the Drawing graphics with offset and zoom subsection of the Drawing graphics section of Chapter 26: Generating graphics.

Note that you can draw characteristics specified or returned in world units (MmeasSetMarker() with M_INCLUSION_POINT_INPUT_UNITS, M_MARKER_REFERENCE_INPUT_UNITS, M_POINT_INPUT_UNITS, M_SEARCH_REGION_INPUT_UNITS, and/or M_RESULT_OUTPUT_UNITS set to M_WORLD). However, if you set any of these constants to M_WORLD and you specify to draw expected marker characteristics from a measurement marker, you must pass **MmeasDraw**() a calibrated target image. Otherwise, the function will generate an error.

Parameters

ContextGraId

Specifies the 2D graphics context to use. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b

2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

MarkerOrResultMeasId

Specifies the identifier of a measurement marker buffer or the identifier of a measurement result buffer. A measurement marker buffer contains both the expected marker characteristics that you specified with MmeasSetMarker(), and the actual marker characteristics that Aurora Imaging Library found in the target image with MmeasFindMarker(). To specify which characteristics to draw (expected or found), use the ControlFlag parameter (M_MARKER or M_RESULT).

If you specify a measurement result buffer, it must contain results from MmeasCalculate().

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Operation

Specifies the type of operation to perform. You can add the possible Operation parameter values in the tables below to draw multiple characteristics at once.

The following operations are available when using a measurement marker buffer (MarkerOrResultMeasId) with either M_MARKER or M_RESULT (ControlFlag), unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For measurement marker buffers with either M_MARKER or M_RESULT																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_INCLUSION_POINT	Draws a cross (+) at the position of the marker's inclusion point. This applies to stripe markers. To specify an inclusion point, use MmeasSetMarker() with M_INCLUSION_POINT. M_DRAW_INCLUSION_POINT is only available with M_MARKER.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION +	Draws a cross (+) at the position of the marker, for each marker occurrence. This applies to edge, stripe, circle, and point markers. For edge, stripe, and circle markers, the found position is drawn (MmeasFindMarker()); you must specify M_RESULT. For point markers, the defined position is drawn (MmeasSetMarker() with M_POSITION), although you can specify either M_MARKER or M_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_SEARCH_DIRECTION	Draws an arrow starting from the middle of the marker's box search region and pointing towards the direction of the edge transition. This applies to edge and stripe markers; in this case, the search direction is based on the marker's orientation (MmeasSetMarker() with M_ORIENTATION).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SEARCH_REGION	Draws the marker's search region. This applies to edge, stripe, and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SEARCH_REGION_CENTER	Draws the center of the marker's search region. This applies to edge, stripe, and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b

The following operations are available when using a measurement marker buffer (MarkerOrResultMeasId) with M_RESULT (ControlFlag).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For measurement marker buffers and M_RESULT																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_EDGES	Draws a line along the found edges of the marker. This applies to edge, stripe, and circle markers. For circle markers, the fitted circle is drawn, along with the found radius and position.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_EDGES_PROFILE +	Draws the first derivative of the intensity profile (edgevalue) from which the marker was established. This applies to edge and stripe markers. If you are drawing in a 2D graphics list, you must add M_DRAW_IN_BOX to this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_EDGEVALUE_MIN_IN_PROFILE +	Draws the threshold above which a grayscale variation (edgevalue) can be considered an edge. This is a graphical representation of MmeasSetMarker() with M_EDGEVALUE_MIN. This applies to edge and stripe markers. If you are drawing in a 2D graphics list, you must add M_DRAW_IN_BOX to this operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_EDGEVALUE_PEAK_WIDTH_IN_PROFILE +	Draws a horizontal line, beginning and ending with arrows (for example, <--->), to indicate the width of the edge peak of an edge or stripe marker. This is a graphical representation of MmeasGetResult() with M_EDGEVALUE_PEAK_WIDTH. This applies to edge and stripe markers. For stripe markers, the width of both its outermost edges is drawn, unless you specify M_EDGE_FIRST or M_EDGE_SECOND. If you are drawing in a 2D graphics list, you must add M_DRAW_IN_BOX to this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION_IN_PROFILE +	Draws a vertical bar at the position of the marker within its profile. This applies to edge and stripe markers. If you are drawing in a 2D graphics list, you must add M_DRAW_IN_BOX to this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SPACING	Draws an H-type line (|-|) indicating the found spacing. This applies to multiple-occurrence markers (edge and stripe).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SUB_POSITIONS +	Draws a cross (+) at the found positions of the subedges. This applies to edge, stripe, and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WIDTH	Draws an H-type line (|-|) to indicate the width of the edge peak of an edge or stripe marker. This is a graphical representation of MmeasGetResult() with M_EDGE_WIDTH. This applies to stripe markers only.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_DRAW_EDGES_PROFILE; M_DRAW_EDGEVALUE_MIN_IN_PROFILE; M_DRAW_EDGEVALUE_PEAK_WIDTH_IN_PROFILE; M_DRAW_POSITION_IN_PROFILE.

You can add the following value to the above-mentioned values to specify whether to scale down (resize) the drawing to fit within the dimensions of the box search region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For scaling down the specified drawing (edge and stripe markers only)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DRAW_IN_BOX	Performs the specified drawing operation (for example, M_DRAW_EDGES_PROFILE), except scaled down (resized) to fit within the dimensions of the box search region. If you do not use this combination value, the same drawing operation is performed (for example, the edge profile is drawn), but sized to fit the dimensions of the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DRAW_POSITION; M_DRAW_EDGEVALUE_PEAK_WIDTH_IN_PROFILE; M_DRAW_SUB_POSITIONS.

You can add one or more of the following values to the above-mentioned values to specify whether to perform the drawing operation on the first or second edge of a stripe.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the edge for which to perform the drawing (stripe markers only)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_FIRST	Draws the specified characteristic for the first outermost edge of a stripe. The first edge of a stripe is the edge closest to the search region's origin.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_SECOND	Draws the specified characteristic for the second outermost edge of a stripe.	c		e	f	g	a		i		j	k	l	m	n	b

The following operation is available when using a measurement result buffer (MarkerOrResultMeasId).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For measurement result buffers and M_RESULT																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_LINE	Draws the line that joins the two markers specified with MmeasCalculate().	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the index of the result to draw from a measurement marker buffer or from a measurement result buffer. When drawing the expected characteristics of a marker (M_MARKER), Aurora Imaging Library does not need any index information and you must set this parameter to M_DEFAULT.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_RESULT_ALL_OCCURRENCES, when drawing the results of a measurement marker buffer or a measurement result buffer. When drawing the expected characteristics of a marker, this parameter must be set to M_DEFAULT, which specifies that the index information is not needed. Note that when drawing the expected characteristics of a marker that is actually a multiple-occurrence marker, only the first instance of the marker is drawn.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_PER_SUBREGION(AIL_INT Subregion, AIL_INT Occurrence)	Specifies that the operation will be performed for a measurement result buffer, according to the specified subregions (one or all) of the specified occurrences (one or all). You can only use M_RESULT_PER_SUBREGION() with edge and stripe markers for the following operations: M_DRAW_SEARCH_DIRECTION, M_DRAW_SEARCH_REGION, M_DRAW_SEARCH_REGION_CENTER, M_DRAW_EDGES_PROFILE, and M_DRAW_EDGEVALUE_MIN_IN_PROFILE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	Subregion Specifies the subregion.														
	M_ALL_SUBREGIONS — Specifies that the operation will be performed for all of the subregion results.														
	Value — Specifies the index of the subregion result for which to perform the operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Occurrence Specifies the occurrence.														

			M_ALL_OCCURRENCES	Specifies that the operation will be performed for all of the result occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
			Value	Specifies the index of the result occurrence for which to perform the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_ALL_OCCURRENCES	Specifies that the operation will be performed for all results.				c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the result to draw.				c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether to perform the drawing operation on the expected marker characteristics, the actual marker characteristics found in the target image, or the results of the measurement calculation. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to use the expected characteristics, the found characteristics, or the calculation results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MARKER	Specifies to use the expected characteristics of a marker (as set with MmeasSetMarker()). In this case, the MarkerOrResultMeasId parameter must be set to the identifier of a measurement marker buffer and the Index parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT	Specifies to use the actual marker characteristics located in the target image (MmeasFindMarker()), or the result of the measurement calculation (MmeasCalculate()). In this case, the MarkerOrResultMeasId parameter must be set to the identifier of either a measurement marker buffer or a measurement result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasFindMarker

Synopsis

Find a marker in a target image and take the specified measurements.

Syntax

```
void MmeasFindMarker(
    AIL_ID ContextId,         //in
    AIL_ID ImageBufId,        //in
    AIL_ID MarkerId,          //in-out
    AIL_INT64 MeasurementList //in
)
```

Description

This function finds an edge, stripe, or circle marker in the specified target image and takes the specified measurements. To locate the marker, Aurora Imaging Library uses its essential characteristics (MmeasSetMarker()), its score characteristics (MmeasSetScore()), and the measurement context (MmeasControl()).

For edge and stripe markers, it is recommended that the box search region be set to an angle close to that of the required marker (that is, within the marker's particular rotation tolerance) to ensure that the marker is found and that results are accurate. The greater the angle of the marker relative to the search region, the greater the distortion of the marker's actual characteristics, which increases the chance that **MmeasFindMarker**() might not successfully locate the marker.

MmeasFindMarker() stores its results in the specified measurement marker (not in a result buffer). To obtain results, use MmeasGetResultSingle() or MmeasGetResult(). All positional results are relative to the top-left pixel in the target image, or the origin of the relative coordinate system if the target is a calibrated image.

If you have associated the target image with a camera calibration context, you can specify that certain input settings be interpreted in world units. To do so, use MmeasSetMarker() with M_MARKER_REFERENCE_INPUT_UNITS, M_INCLUSION_POINT_INPUT_UNITS, and/or M_SEARCH_REGION_INPUT_UNITS set to M_WORLD. By using a calibrated image, you can specify the search region according to the relative coordinate system using MmeasSetMarker(), and then fixture the relative coordinate system to each object. This way, you will only need to place the search region once, and all of the search results will be made from the same reference location. Note that if you set any of these constants to M_WORLD but you don't pass **MmeasFindMarker**() a calibrated target image, the function will generate an error.

For edge and stripe markers, you can limit the find operation to a region of the target image buffer using a rectangular region of interest (ROI) set with MbufSetRegion(). For circle markers, you can limit the find operation to a region of the target image buffer using a circular region of interest (ROI) set with MbufSetRegion(). The find marker operation will then search inside the ROI instead of inside any previously set search regions. Note that while the region of interest (ROI) affects the marker occurrences, it will not change the settings of any previously set search regions. In these cases, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER).

When finding an edge or stripe marker, the target image buffer can have a rectangular region of interest (ROI), defined using MbufSetRegion() (with or without rotation). When finding a circle marker, the target image buffer can have a circular region of interest (ROI), defined using MbufSetRegion().

Parameters

ContextId

Specifies the identifier of the measurement context.

For specifying the measurement context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default measurement context of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
Measurement context ID	Specifies a valid measurement context, previously allocated using MmeasAllocContext().	c		e	f	g	a		i		j	k	l	m	n	b

ImageBufId

Specifies the identifier of the target image buffer in which to locate the marker. This buffer must be a 1-band, 8-bit or 16-bit unsigned image buffer.

MarkerId

Specifies the identifier of the marker to be located in the image.

MeasurementList

Specifies the measurements to perform.

Aurora Imaging Library considers the specified measurements when establishing results (MmeasGetResult() or MmeasGetResultSingle()). If you do not specify M_DEFAULT, which takes all possible measurements applicable to the marker type, you might not be able to retrieve all results for that marker. For the vast majority of applications, M_DEFAULT should be set.

Note that if you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to perform certain measurements, such as determining the equation of a line (M_LINE_EQUATION). To ensure that Aurora Imaging Library does not internally use multiple subregions when you did not specify any, you must not select measurements that require them.

> Except for M_DEFAULT, you can add measurement list values together (for example, M_POSITION + M_ANGLE + M_LENGTH).

The measurements in the following table can be performed with edge, stripe, or circle markers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurements to perform when using edge, stripe, or circle markers																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs all measurements that apply to the marker type. For example, if you are using a circle, Aurora Imaging Library takes all measurements possible for circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION +	Determines the position of the marker. By specifying M_POSITION, all of the following will also be accessible with MmeasGetResult() or MmeasGetResultSingle() (for edge and stripe markers): M_BOX_EDGEVALUES, M_EDGE_END, M_EDGE_STRENGTH, M_EDGE_START, M_ORIENTATION, M_EDGEVALUE_PEAK_POS_MAX, M_EDGEVALUE_PEAK_POS_MIN, and M_SPACING.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for M_POSITION.

You can use one or more of the following values in combination with each other or with the above-mentioned value,

to specify the measurement to perform when using edge or stripe markers.

The measurements in the following table can be performed with edge or stripe markers, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurements to perform when using edge or stripe markers																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ANGLE	Determines the angle of the mean line of a marker, relative to the output coordinate system specified using MmeasControl() with M_RESULT_OUTPUT_UNITS. The angle can result in a value between 0° and 360°. For a stripe marker, Aurora Imaging Library establishes the angle of three lines: the stripe's mean line, the stripe's first outermost edge, and the stripe's second outermost edge. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. If you did not explicitly specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to establish the lines it requires to determine M_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_CONTRAST	Determines the grayscale difference between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established. For a stripe marker, Aurora Imaging Library establishes this value from the stripe's first and second outermost edges.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_INSIDE	Determines the number of edges located between the two exterior edges of a stripe. This only applies to stripe markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_WIDTH	Determines the distance between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established. For a stripe marker, Aurora Imaging Library establishes this value for the stripe's first and second outermost edges.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_PEAK_CONTRAST	Determines the grayscale difference of the intensity transition between the first zero edgevalues on both sides of the edge peak (before M_EDGEVALUE_PEAK_POS_MIN, and after M_EDGEVALUE_PEAK_POS_MAX). For a stripe marker, Aurora Imaging Library establishes this value for the stripe's first and second outermost edges. Note that some images might never have a zero edgevalue, in which case you should use M_EDGE_CONTRAST for a more meaningful result.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_WIDTH	Determines the distance between the minimum (M_EDGEVALUE_PEAK_POS_MIN) and maximum (M_EDGEVALUE_PEAK_POS_MAX) positions of the edge peak (that is, the minimum and maximum position of the edgevalue along the first derivative representation of the intensity). For a stripe marker, Aurora Imaging Library establishes this value for the stripe's first and second outermost edges.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH	Determines the length of the side of the marker perpendicular to the search direction. This is useful to retrieve the number of pixels that were projected into a single value of the intensity profile without having to find M_ORIENTATION first.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_END_POINT_FIRST	Determines the X- and Y-coordinates of the first intersection point between the marker's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to establish the line it requires to determine M_LINE_END_POINT_FIRST.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_END_POINT_SECOND	Determines the X- and Y-coordinates of the second intersection point between the marker's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to establish the line it requires to determine M_LINE_END_POINT_SECOND.	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_EQUATION	Determines the equation of the line following an edge. The line equation is of the general form, $Ax + By + C = 0$. For a stripe marker, Aurora Imaging Library establishes the equation of three lines: the stripe's mean line, the stripe's first edge, and the stripe's second edges. If you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to establish the lines it requires to determine M_LINE_EQUATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_POLARITY	Determines whether the edge(s) are rising (positive polarity) or falling (negative polarity).	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE_WIDTH	Determines the width of the stripe marker. The stripe's width refers to the distance between the position of the maximum edgevalue of the stripe's two outermost edges. This only applies to stripe markers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value that can be used alone or as combination value for M_POSITION.

You can use the following value on its own, or add it to the above-mentioned value, to specify the measurement to perform when using circle markers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurement to perform when using circle markers																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RADIUS	Determines the distance between the circle's circumference and the circle's center.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasFree

Synopsis

Free a measurement context, marker buffer, or result buffer.

Syntax

```
void MmeasFree(
    AIL_ID MeasId //in
)
```

Description

This function deletes the specified marker buffer, result buffer, or context identifier and releases any memory associated with it, unless M_UNIQUE_ID was specified during allocation.

Parameter

MeasId

Specifies the measurement identifier to free. The identifier must have been successfully allocated with MmeasAllocMarker(), MmeasAllocResult(), or MmeasAllocContext() prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasGetResult

Synopsis

Get the specified type of result(s) for all points, edges, stripes, or circles from a measurement marker buffer or a measurement result buffer.

Syntax

```
void MmeasGetResult(
    AIL_ID MarkerOrMeasResultId, //in
    AIL_INT64 ResultType,        //in
    void *FirstResultArrayPtr,   //out
    void *SecondResultArrayPtr   //out
)
```

Description

This function retrieves the result(s) of the specified type for all points, edges, stripes, or circles from a measurement marker buffer or a measurement result buffer. Measurement marker buffers hold the results of an MmeasFindMarker() operation, while measurement result buffers hold the results of an MmeasCalculate() operation. Depending on the buffer you are using, you must call either MmeasFindMarker() or MmeasCalculate() prior to calling **MmeasGetResult**(); otherwise, you will get incorrect results.

If you are using a multiple-occurrence marker, **MmeasGetResult**() typically returns the specified result for all occurrences. The result for the marker's first occurrence will be followed by the result for the marker's second occurrence, and so on. To return results for a single occurrence, use MmeasGetResultSingle(). Note that some results cannot be returned for each occurrence since they are based on the marker itself (for example, M_BOX_ANGLE_FOUND, which returns the angle of the marker's box search region); in such cases, you will get a single result, even if you are using a multiple-occurrence marker.

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

If your target image was associated with a camera calibration context and you want to retrieve positional and dimensional results in pixel units, use MmeasSetMarker() for a measurement marker buffer, or MmeasControl() for a measurement result buffer, with M_RESULT_OUTPUT_UNITS set to M_PIXEL. If you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MmeasGetResult**() will generate an error.

For certain result types, Aurora Imaging Library returns one value for an edge marker, and two values for a stripe marker. In these cases, for an edge marker, Aurora Imaging Library returns all results to FirstResultArrayPtr; SecondResultArrayPtr must be set to **M_NULL**.

When retrieving a result for a stripe marker, result types can return one general result for the stripe, a result for both edges of the stripe, or a result for one edge of the stripe. To return the result for one edge of the stripe, you must specify the edge using the combination value M_EDGE_FIRST or M_EDGE_SECOND.

When retrieving non-positional results for both edges of a stripe marker, Aurora Imaging Library returns the results for the first edge to FirstResultArrayPtr and for the second edge to SecondResultArrayPtr. When retrieving non-positional results for one edge of the stripe, Aurora Imaging Library always returns the result to FirstResultArrayPtr.

When retrieving positional results for a stripe, Aurora Imaging Library returns the results for the X-coordinate of the stripe to FirstResultArrayPtr and the Y-coordinate of the stripe to SecondResultArrayPtr. When retrieving positional results for one edge of the stripe, Aurora Imaging Library returns the X-coordinate to FirstResultArrayPtr and the Y-coordinate to SecondResultArrayPtr. You can set FirstResultArrayPtr or SecondResultArrayPtr to **M_NULL** to retrieve only one coordinate.

Parameters

MarkerOrMeasResultId

Specifies the identifier of the measurement marker buffer (allocated with MmeasAllocMarker()) or measurement result buffer (allocated with MmeasAllocResult()) from which to retrieve results.

ResultType

Specifies the type of result(s) to retrieve.

See the Parameter associations section for possible values that can be specified.

FirstResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the first array in which to write the requested information.

With a multiple-occurrence marker, the results for each occurrence of the marker will be stored in a separate element of the specified arrays.

See the Parameter associations section for possible values that can be returned.

SecondResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64
- M_NULL

Specifies the address of the second array in which to write the requested information.

For most result types, only FirstResultArrayPtr will be used, and SecondResultArrayPtr should be set to **M_NULL**.

With a multiple-occurrence marker, results for each edge or stripe are held in an array. The results for each occurrence of the marker will be stored in a separate element of the specified arrays.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **FirstResultArrayPtr** and **SecondResultArrayPtr** parameters.

- For any type of marker (measurement marker buffer) or a measurement result buffer
- For an edge or stripe marker (measurement marker buffer) or a measurement result buffer
- For any type of marker (measurement marker buffer)

- For an edge, stripe, or circle marker (measurement marker buffer)
- For an edge or stripe marker (measurement marker buffer)
- For a circle marker (measurement marker buffer)
- For a measurement result buffer

To retrieve results from any type of marker (measurement marker buffer), or from a measurement result buffer, ResultType can be set to one of the values specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type of marker (measurement marker buffer) or a measurement result buffer																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_NUMBER +	Retrieves the number of points, edges, stripes, or circles measured. Note that edges, stripes, and circles are searched for, but points are placed manually. For circle markers, M_NUMBER will never return a value greater than one, since circles cannot be defined as a multiple-occurrence marker. If you are getting results from a measurement result buffer, and you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()). After a call to the MmeasFindMarker() function, this number is equal to the number of occurrences of a marker found in the search region. After a call to the MmeasCalculate() function, this number is equal to the smallest number of occurrences held in either marker involved in the calculation. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of points, edges, stripes, or circles measured.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from an edge or stripe marker (measurement marker buffer), or from a measurement result buffer, ResultType can be set to one of the values specified in the table below. If you are getting results from a measurement result buffer, and you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an edge or stripe marker (measurement marker buffer) or a measurement result buffer																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	FirstResultArrayPtr - Possible values returned															
	SecondResultArrayPtr - Possible values returned															
M_ANGLE +		Retrieves the angle for each marker occurrence, in degrees. The angle is relative to the output coordinate system specified using M_RESULT_OUTPUT_UNITS with either MmeasSetMarker() (for a measurement marker buffer) or MmeasControl() (for a measurement result buffer). If retrieved from a measurement marker buffer, M_ANGLE returns the angle of the line measured for the edge. For a stripe marker, this line follows the center of the stripe and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the angle of the first or second edge is returned, respectively. If retrieved from a measurement result buffer, M_ANGLE returns the angle of the lines joining each of the occurrences of the two markers, relative to the positive X-axis. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.														
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_A +		Retrieves the coefficient *A* of the line equation, for each marker occurrence. The line equation is of the general form, $Ax + By + C = 0$. If retrieved from an edge marker, M_LINE_A returns the coefficient *A* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *A* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_A returns the coefficient *A* of the line joining each occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_B +	Retrieves the coefficient *B* of the line equation, for each marker occurrence. The line equation is of the general form, *Ax* + *By* + *C* = 0. If retrieved from an edge marker, M_LINE_B returns the coefficient *B* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *B* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_B returns the coefficient *B* of the line joining each occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_C +	Retrieves the coefficient *C* of the line equation, for each marker occurrence. The line equation is of the general form, *Ax* + *By* + *C* = 0. If retrieved from an edge marker, M_LINE_C returns the coefficient *C* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *C* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_C returns the coefficient *C* of the lines joining each occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from any type of marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type of marker (measurement marker buffer)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																

M_POSITION +	Retrieves the X- and Y-coordinates of the position, for each marker occurrence. For point markers, the position refers to the X- and Y-coordinates specified using MmeasSetMarker() with M_POSITION. For edge markers, the position refers to the X- and Y-coordinates of the edge's maximum edgevalue (highest edge peak) along the center of the search region. For stripe markers, the position refers to the X- and Y-coordinates at the center of a theoretical line between the position (maximum edgevalue) of the stripe's two outermost edges. If you use M_EDGE_FIRST or M_EDGE_SECOND, you will get the X- and Y-coordinates of that edge's maximum edgevalue (highest edge peak). For circle markers, the position refers to the X- and Y-coordinates at the circle's center.															
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALID_FLAG +	Retrieves whether the marker was found, or that the coordinates of a point marker (MmeasAllocMarker() with M_POINT) are set to a valid position. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the marker was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the marker was found.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from an edge, stripe, or circle marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an edge, stripe, or circle marker (measurement marker buffer)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																

	SecondResultArrayPtr - Possible values returned															
M_FIT_ERROR_MAX +		Retrieves the maximum distance from a subedge to the fitted line equation (for edge or stripe markers) or to the fitted circle (for circle markers), for each marker occurrence. This distance is called the maximum fit error. If you did not specify subregions (for edges or stripes), M_FIT_ERROR_MAX returns 0. Note that circles always have subregions.														
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the maximum fit error. For a stripe marker, this is the maximum fit error for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the maximum fit error for the second edge of a stripe. For edge and circle markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_OUTLIERS +		Retrieves the number of subedges considered outliers, for each marker occurrence. Outliers are subedges within the search region that do not contribute to marker's fit operation since they are considered to be too far away from the general distribution of subedges. The number of outliers corresponds to the number of fitted subedges subtracted from the number of subregions. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the number of outliers for the edge or circle. For a stripe marker, this is the number of outliers for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the number of outliers for the second edge of a stripe. For edge and circle markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_REGION_WAS_CLIPPED +	Retrieves whether the marker's search region was clipped. This value is only useful if you enabled clipping (MmeasSetMarker() with M_SEARCH_REGION_CLIPPING). Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the search region was not clipped.	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies that the search region was clipped.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_EDGES_MARKER_INDEX +	Retrieves the index of the occurrence on which each subedge is located, for each marker occurrence. If a subedge is not found in a subregion, an arbitrary number is returned. Use M_SUB_EDGES_WEIGHT to determine if a subedge was actually found. This result is useful when retrieving subedge-type of results for a multiple-occurrence marker. For example, you can use the index to separate, by marker, the values obtained with M_SUB_EDGES_POSITION and M_SUB_EDGES_WEIGHT.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the index values. For a stripe marker, these are the index values for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns whether the result refers to the first or second edge of the stripe. For edge and circle markers, this parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_FIRST	Specifies that the edge is the first edge in the stripe.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_SECOND	Specifies that the edge is the second edge in the stripe.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUB_EDGES_POSITION +	Retrieves the X- and Y-coordinates of the subedges, for each marker occurrence. If a subedge is not found in a subregion, the X- and Y- positions will be arbitrary numbers. Use M_SUB_EDGES_WEIGHT to determine whether a subedge was found. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_EDGES_WEIGHT +	Retrieves the weight of the subedges, for each marker occurrence. The weight indicates whether a subedge was found in the subregion. Found subedges are used to calculate the fit, angle, and line equation values. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the subedge was not found.	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies that the subedge was found.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_NUMBER_USED +	Retrieves the number of subregions used. Note that if you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to perform certain measurements.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of subregions.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from an edge or stripe marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below, unless otherwise specified.

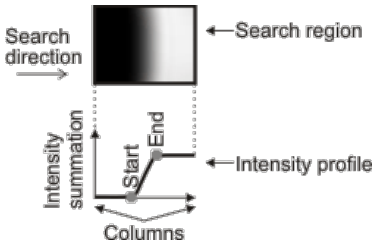
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

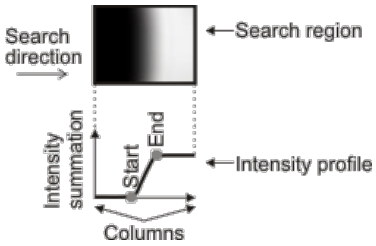
For an edge or stripe marker (measurement marker buffer)																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_BOX_ANGLE_FOUND +	Retrieves the angle found for the box search region when the angle is internally determined by Aurora Imaging Library. In this case, you must have specified M_ANY as the angle of the box search region (MmeasSetMarker() with M_BOX_ANGLE) or enabled an angular search (M_BOX_ANGLE_MODE set to M_ENABLE). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the angle of the box search region, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_BOTTOM_LEFT +	Retrieves the coordinates of the bottom-left corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account. Note that this result type always returns one set of values (X- and Y-positions), even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-position.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-position.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOX_CORNER_BOTTOM_RIGHT +	Retrieves the coordinates of the bottom-right corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account. Note that this result type always returns one set of values (X- and Y-positions), even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-position.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-position.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_TOP_LEFT +	Retrieves the coordinates of the top-left corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account. Note that this result type always returns one set of values (X- and Y-positions), even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-position.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-position.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_TOP_RIGHT +	Retrieves the coordinates of the top-right corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account. Note that this result type always returns one set of values (X- and Y-positions), even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-position.	c		e	f	g	a		i		j	k	l	m	n	b

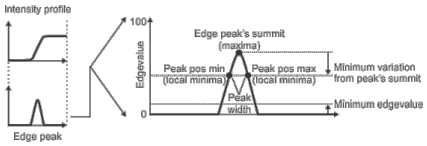
	SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-position.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_EDGEVALUES +		Retrieves the edgevalues of the marker's box search region. Aurora Imaging Library establishes edgevalues by applying a first derivative filter to the search region's intensity profile and normalizing the output according to the number of pixels and the maximum pixel value possible. To establish the intensity profile, Aurora Imaging Library projects the pixels bounded by the two-dimensional search region (or each subregion) into a one-dimensional pixel intensity summation, which is performed vertically or horizontally, depending on the search region's origin and search direction. To obtain the edgevalues of each subregion, see M_SUB_REGIONS_EDGE_VALUES.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edgevalues.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_EDGEVALUES_NUMBER +		Retrieves the number of edgevalues that M_BOX_EDGEVALUES returns. This value corresponds to the number of pixels along the width or height of the marker's box search region, depending on the search region's origin and search direction. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of edgevalues.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_FROM_BOX_ORIGIN +		Retrieves the position, as a distance value relative to the origin of the box search region, for each marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position as a distance value. For a stripe marker, this is the distance to the first edge of the stripe.	c		e	f	g	a		i		j	k	l	m	n	b

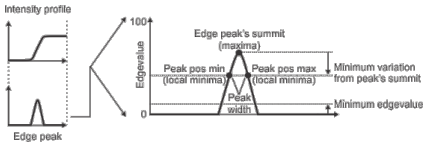
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the position as the distance to the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PROFILE_SCALE_OFFSET +		Retrieves the scale and offset used to draw a profile.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the scale used to draw a profile. If M_AUTO_SCALE_PROFILE was set using MmeasSetMarker(), this returns the resolved value.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the offset used to draw a profile. If M_AUTO_OFFSET_PROFILE was set using MmeasSetMarker(), this returns the resolved value.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_CONTRAST +		Retrieves the grayscale difference between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established, for each marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the contrast. For a stripe marker, this is the contrast of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the contrast of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_END +	Retrieves the X- and Y-coordinates of the end of the edge (that is, the end of the intensity transition from which the edge is established), for each marker occurrence. For a stripe marker, the coordinates representing the average value of end positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. In the intensity profile calculated for a search region, M_EDGE_START represents the first significant slope used to establish the edge, while M_EDGE_END represents the first significant slope in the opposite direction used to establish the edge. Depending on the search region's origin and search direction, the edge's start and end can be on either side of the edge.  To retrieve the minimum and maximum position of the edge peak of the marker, use M_EDGEVALUE_PEAK_POS_MIN and M_EDGEVALUE_PEAK_POS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the end of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the end of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_INSIDE +	Retrieves the number of edges located between the two exterior edges of a stripe, for each marker occurrence. This result only applies to stripes.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_START +	Retrieves the X- and Y-coordinates of the start of the edge (that is, the start of the intensity transition from which the edge is established), for each marker occurrence. For a stripe marker, the coordinates representing the average value of start positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. In the intensity profile calculated for a search region, M_EDGE_START represents the first significant slope used to establish the edge, while M_EDGE_END represents the first significant slope in the opposite direction used to establish the edge. Depending on the search region's origin and search direction, the edge's start and end can be on either side of the edge.  To retrieve the minimum and maximum position of the edge peak of the marker, use M_EDGEVALUE_PEAK_POS_MIN and M_EDGEVALUE_PEAK_POS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the start of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the start of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_STRENGTH +	Retrieves the greatest edgevalue of the edge, for each marker occurrence. The greatest edgevalue of an edge is considered to be its edge strength.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edge strength. For a stripe marker, this is the strength of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b

	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the strength of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_WIDTH +		Retrieves the distance between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established, for each marker occurrence. To retrieve the width of the edge established by the edge peak of the marker, use M_EDGEVALUE_PEAK_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edge width. For a stripe marker, this is the edge width of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the edge width of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_CONTRAST +		Retrieves the grayscale difference of the intensity transition between the first zero edgevalues on both sides of the established edge peak (before M_EDGEVALUE_PEAK_POS_MIN, and after M_EDGEVALUE_PEAK_POS_MAX), for each marker occurrence. Note that some images might never have a zero edgevalue, in which case you should use M_EDGE_CONTRAST for a more meaningful result.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the contrast. For a stripe marker, this is the contrast of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the contrast of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_PEAK_POS_MAX +	Retrieves the X- and Y-coordinates of the maximum value of the edge peak (that is, the maximum edgevalue along the first derivative representation of the intensity profile), known as the summit, for each marker occurrence. For a stripe marker, the coordinates representing the average value of maximum positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. To establish the maximum and minimum positions of the edge peak, Aurora Imaging Library considers the portion of the peak that contains the maximum edgevalue, and that satisfies the M_EDGEVALUE_MIN and M_EDGEVALUE_VAR_MIN constraints set using MmeasSetMarker(). That is, the peak must dip, on both sides, below either the minimum edgevalue threshold (M_EDGEVALUE_MIN) or the local edgevalue threshold established by subtracting M_EDGEVALUE_VAR_MIN from the peak's summit, whichever is highest.  Depending on the search region's origin and search direction, the maximum and minimum position can be on either side of the edge peak. Note that you can also retrieve different start and end positions of the edge using M_EDGE_START and M_EDGE_END.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the maximum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the maximum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_PEAK_POS_MIN +	Retrieves the X- and Y-coordinates of the minimum position of the edge peak (that is, the minimum edgevalue along the first derivative representation of the intensity profile), for each marker occurrence. For a stripe marker, the coordinates representing the average value of minimum positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. To establish the minimum and maximum positions of the edge peak, Aurora Imaging Library considers the portion of the peak that contains the maximum edgevalue, and that satisfies the M_EDGEVALUE_MIN and M_EDGEVALUE_VAR_MIN constraints set using MmeasSetMarker(). That is, the peak must dip, on both sides, below either the minimum edgevalue threshold (M_EDGEVALUE_MIN)or the local edgevalue threshold established by subtracting M_EDGEVALUE_VAR_MIN from the peak's summit, whichever is highest.  Depending on the search region's origin and search direction, the maximum and minimum position can be on either side of the edge peak's summit. Note that you can also retrieve different start and end positions of the edge using M_EDGE_START and M_EDGE_END.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the minimum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the minimum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_WIDTH +	Retrieves the distance between the minimum (M_EDGEVALUE_PEAK_POS_MIN) and maximum (M_EDGEVALUE_PEAK_POS_MAX) position of the edge peak (that is, the minimum and maximum positions of the edgevalue along the first derivative representation of the intensity profile), for each marker occurrence. To retrieve the width of the edge instead of the width of the edge peak, use M_EDGE_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b

	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edge width. For a stripe marker, Aurora Imaging Library returns the edge width of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the edge width of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +		Retrieves the length of the side of the search region perpendicular to the search direction. This is useful to retrieve the number of pixels that were projected into a single value of the intensity profile without having to find M_ORIENTATION first.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_END_POINT_FIRST +		Retrieves the X- and Y-coordinates of the first intersection point, for each marker occurrence, between the edge's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you request this result for a specific edge of a stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, M_LINE_END_POINT_FIRST will return the first intersection point of the line following the specified edge and the box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the intersection will take that rotation into account. For vertical edges, the first intersection point is typically at the top of the box search region, while for horizontal edges the first intersection point is typically on the right of the box search region. Note that the intersection point that is considered to be the first or second (M_LINE_END_POINT_SECOND) can change depending on the edge's angle and whether the box search region is rotated.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b

SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_END_POINT_SECOND +	Retrieves the X- and Y-coordinates of the second intersection point, for each marker occurrence, between the edge's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you request results for a specific edge of a stripe marker occurrence, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, M_LINE_END_POINT_SECOND returns the second intersection point of the line following the specified edge and the box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the intersection will take that rotation into account. For vertical edges, the second intersection point is typically at the bottom of the box search region, while for horizontal edges the second intersection point is typically on the left of the box search region. Note that the intersection point that is considered to be the first (M_LINE_END_POINT_FIRST) or second can change depending on the edge's angle and whether the box search region is rotated.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the X-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the Y-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIENTATION +	Retrieves the orientation of the marker. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL	Specifies a horizontal orientation.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL	Specifies a vertical orientation.	c		e	f	g	a		i		j	k	l	m	n	b

SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_POLARITY +	Retrieves the polarity, for each marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the polarity of the edge. For a stripe marker, this is the polarity of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies that the polarity of the edge is negative.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies that the polarity of the edge is positive.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *Set this parameter to M_NULL* Returns the polarity of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies that the polarity of the second edge is negative.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies that the polarity of the second edge is positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +	Retrieves the score, for each marker occurrence. This score represents how confident you should be that Aurora Imaging Library found what you expected. For an edge marker, the score is the product of all score characteristics set for the edge. For a stripe marker, the score is the product of the global score of both edges, and of all the characteristics for the stripe.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_TOTAL +	Retrieves the average score of all marker occurrences. This score represents how confident you should be that Aurora Imaging Library found what you expected. Note that this result type always returns one value, even when using a multiple-occurrence marker, and there are multiple occurrences. Use M_SCORE to get the score of a single occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING +		Retrieves the distance between each marker occurrence and the next.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE_WIDTH +		Retrieves the width of the stripe, for each marker occurrence. The stripe's width refers to the distance between the position of the maximum edgevalue of the stripe's two outermost edges. This result is only available for stripe markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_EDGE_VALUES +		Retrieves the edgevalues in all subregions of the marker's box search region. Aurora Imaging Library establishes edgevalues by applying a first derivative filter to the search region's intensity profile and normalizing the output according to the number of pixels and the maximum pixel value possible. To establish the intensity profile, Aurora Imaging Library projects the pixels bounded by the two-dimensional search subregion into a one-dimensional pixel intensity summation, which is performed vertically or horizontally, depending on the search region's origin and search direction.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edgevalues. The edgevalues are copied into a concatenated array.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For an edge or stripe marker (measurement marker buffer) or a measurement result buffer; and for the following values: M_POSITION; M_FIT_ERROR_MAX; M_NUMBER_OF_OUTLIERS; M_SUB_EDGES_MARKER_INDEX; M_SUB_EDGES_POSITION; M_SUB_EDGES_WEIGHT; M_DISTANCE_FROM_BOX_ORIGIN; M_EDGE_CONTRAST; M_EDGE_END; M_EDGE_START; M_EDGE_STRENGTH; M_EDGE_WIDTH; M_EDGEVALUE_PEAK_CONTRAST; M_EDGEVALUE_PEAK_POS_MAX; M_EDGEVALUE_PEAK_POS_MIN; M_EDGEVALUE_PEAK_WIDTH; M_LENGTH; M_LINE_END_POINT_FIRST; M_LINE_END_POINT_SECOND; M_POLARITY; M_SPACING.

You can add one of the following values to the above-mentioned values to specify the edge for which to retrieve results (stripe markers only).

Note that when retrieving non-positional results, if you explicitly request the result for a specific edge of a stripe, using M_EDGE_FIRST or M_EDGE_SECOND, you must retrieve the result for that edge with FirstResultArrayPtr. In this case, you must set SecondResultArrayPtr to **M_NULL**. For positional results, you can set FirstResultArrayPtr or SecondResultArrayPtr to **M_NULL** if you only want to retrieve the X- or Y-coordinates, respectively. You must always pass at least one pointer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the edge for which to retrieve results (stripe markers only)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_FIRST	Retrieves the specified result for the first outermost edge of a stripe.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_SECOND	Retrieves specified result for the second outermost edge of a stripe.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from a circle marker (measurement marker buffer), ResultType can be set to the value specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a circle marker (measurement marker buffer)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_RADIUS +	Retrieves the circle's radius.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from a measurement result buffer, the ResultType parameter can be set to one of the following values. If you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement result buffer																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																

M_DISTANCE +	Retrieves the distance between each occurrence of the two markers.															
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_X +	Retrieves the distance on the X-axis between each occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_Y +	Retrieves the distance on the Y-axis between each occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For an edge or stripe marker (measurement marker buffer) or a measurement result buffer, For a measurement result buffer; and for the following values: M_POSITION; M_FIT_ERROR_MAX; M_NUMBER_OF_OUTLIERS; M_SUB_EDGES_MARKER_INDEX; M_SUB_EDGES_POSITION; M_SUB_EDGES_WEIGHT; M_BOX_EDGEVALUES; M_DISTANCE_FROM_BOX_ORIGIN; M_EDGE_CONTRAST; M_EDGE_END; M_EDGE_START; M_EDGE_STRENGTH; M_EDGE_WIDTH; M_EDGEVALUE_PEAK_CONTRAST; M_EDGEVALUE_PEAK_POS_MAX; M_EDGEVALUE_PEAK_POS_MIN; M_EDGEVALUE_PEAK_WIDTH; M_LENGTH; M_LINE_END_POINT_FIRST; M_LINE_END_POINT_SECOND; M_POLARITY; M_SCORE; M_SPACING; M_STRIPE_WIDTH; M_SUB_REGIONS_EDGE_VALUES.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For an edge or stripe marker (measurement marker buffer) or a measurement result buffer, For a measurement result buffer; and for the following values: M_POSITION; M_FIT_ERROR_MAX; M_NUMBER_OF_OUTLIERS; M_DISTANCE_FROM_BOX_ORIGIN; M_EDGE_CONTRAST; M_EDGE_END; M_EDGE_INSIDE; M_EDGE_START; M_EDGE_STRENGTH; M_EDGE_WIDTH;

M_EDGEVALUE_PEAK_CONTRAST; M_EDGEVALUE_PEAK_POS_MAX; M_EDGEVALUE_PEAK_POS_MIN; M_EDGEVALUE_PEAK_WIDTH; M_LENGTH; M_LINE_END_POINT_FIRST; M_LINE_END_POINT_SECOND; M_POLARITY; M_SCORE; M_SPACING; M_STRIPE_WIDTH.

You can add one of the following values to the above-mentioned values to get the type of statistic to be performed.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of statistic to be performed																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_MAX	Retrieves the maximum value for the requested characteristic for all the occurrences of a marker. Only one result is returned.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Retrieves the mean value for the requested characteristic for all the occurrences of a marker. Only one result is returned.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Retrieves the minimum value for the requested characteristic for all the occurrences of a marker. Only one result is returned.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STANDARD_DEVIATION	Retrieves the standard deviation of the values for the requested characteristic for all the occurrences of a marker. Only one result is returned.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For any type of marker (measurement marker buffer) or a measurement result buffer
- For an edge or stripe marker (measurement marker buffer) or a measurement result buffer
- For any type of marker (measurement marker buffer)
- For an edge, stripe, or circle marker (measurement marker buffer)
- For an edge or stripe marker (measurement marker buffer)
- For a circle marker (measurement marker buffer)
- For a measurement result buffer

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																		
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasGetResultSingle

Synopsis

Get the specified type of result for a specific point, edge, stripe, or circle from a measurement marker buffer or a measurement result buffer.

Syntax

```
void MmeasGetResultSingle(
    AIL_ID MarkerOrMeasResultId, //in
    AIL_INT64 ResultType,        //in
    void *FirstResultArrayPtr,   //out
    void *SecondResultArrayPtr,  //out
    AIL_INT ResultIndex          //in
)
```

Description

This function retrieves the specified result of the specified type for a point, edge, stripe, or circle from a measurement marker buffer or a measurement result buffer. Measurement marker buffers hold the results of an MmeasFindMarker() operation, while measurement result buffers hold the results of an MmeasCalculate() operation. Depending on the buffer you are using, you must call either MmeasFindMarker() or MmeasCalculate() prior to calling MmeasGetResult(); otherwise, you will get an incorrect result.

If you are using a multiple-occurrence marker, **MmeasGetResultSingle**() returns the specified result for a single occurrence, which you must specify with the ResultIndex parameter. To return results for all occurrences, use MmeasGetResult().

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

If your target image was associated with a camera calibration context and you want to retrieve positional and dimensional results in pixel units, use MmeasSetMarker() for a measurement marker buffer, or MmeasControl() for a measurement result buffer, with M_RESULT_OUTPUT_UNITS set to M_PIXEL. If you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MmeasGetResultSingle**() will generate an error.

For certain result types, Aurora Imaging Library returns one value for an edge marker, and two values for a stripe marker. In these cases, for an edge marker, Aurora Imaging Library returns the result to FirstResultArrayPtr; SecondResultArrayPtr must be set to **M_NULL**.

When retrieving a result for a stripe marker, result types can return one general result for the stripe, a result for both edges of the stripe, or a result for one edge of the stripe. To return the result for one edge of the stripe, you must specify the edge using the combination value M_EDGE_FIRST or M_EDGE_SECOND.

When retrieving non-positional results for both edges of a stripe marker, Aurora Imaging Library returns the results for the first edge to FirstResultArrayPtr and for the second edge to SecondResultArrayPtr. When retrieving non-positional results for one edge of the stripe, Aurora Imaging Library always returns the result to FirstResultArrayPtr.

When retrieving positional results for a stripe, Aurora Imaging Library returns the results for the X-coordinate of the stripe to FirstResultArrayPtr and the Y-coordinate of the stripe to SecondResultArrayPtr. When retrieving positional results for one edge of the stripe, Aurora Imaging Library returns the X-coordinate to FirstResultArrayPtr and the Y-coordinate to SecondResultArrayPtr. You can set FirstResultArrayPtr or SecondResultArrayPtr to **M_NULL** to retrieve only one coordinate.

Parameters

MarkerOrMeasResultId

Specifies the identifier of the measurement marker (allocated with MmeasAllocMarker()) or measurement result buffer (allocated with MmeasAllocResult()) from which to retrieve results.

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

FirstResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the first variable or array in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

SecondResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64
- M_NULL

Specifies the address of the second variable or array in which to write the requested information.

For most result types, only FirstResultArrayPtr will be used and SecondResultArrayPtr should be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

ResultIndex

Specifies the index of the occurrence for which to retrieve information.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* < *NumberOfOccurrences*	Specifies the index of the occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the

FirstResultArrayPtr and **SecondResultArrayPtr** parameters.

- For any type of marker (measurement marker buffer) or a measurement result buffer
- For an edge or stripe marker (measurement marker buffer) or a measurement result buffer
- For any type marker (measurement marker buffer)
- For an edge, stripe, or circle marker (measurement marker buffer)
- For an edge or stripe marker (measurement marker buffer)
- For a circle marker (measurement marker buffer)
- For a measurement result buffer

To retrieve a result from any type of marker (measurement marker buffer), or from a measurement result buffer, ResultType can be set to one of the values specified in the table below. If you are getting results from a measurement result buffer, and you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type of marker (measurement marker buffer) or a measurement result buffer																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstResultArrayPtr - Possible values returned																
	SecondResultArrayPtr - Possible values returned																
M_NUMBER +		Retrieves the number of points, edges, stripes, or circles measured. Note that edges, stripes, and circles are searched for, but points are placed manually. For circle markers, M_NUMBER will never return a value greater than one, since circles cannot be defined as a multiple-occurrence marker. After a call to the MmeasFindMarker() function, this number is equal to the number of occurrences of a marker found in the search region. After a call to the MmeasCalculate() function, this number is equal to the smallest number of occurrences held in either marker involved in the calculation.															
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of points, edges, stripes, or circles measured.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from an edge or stripe marker (measurement marker buffer), or from a measurement result buffer, ResultType can be set to one of the values specified in the table below. If you are getting results from a measurement result buffer, and you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an edge or stripe marker (measurement marker buffer) or a measurement result buffer																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	FirstResultArrayPtr - Possible values returned																
	SecondResultArrayPtr - Possible values returned																
M_ANGLE +		Retrieves the angle of the edge for the marker occurrence, in degrees. The angle is relative to the output coordinate system specified using M_RESULT_OUTPUT_UNITS with either MmeasSetMarker() (for a measurement marker buffer) or MmeasControl() (for a measurement result buffer). If retrieved from a measurement marker buffer, M_ANGLE returns the angle of the line measured for the edge. For a stripe marker, this line follows the center of the stripe and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the angle of the first or second edge is returned, respectively. If retrieved from a measurement result buffer, M_ANGLE returns the angle of the line joining the two marker occurrences, relative to the positive X-axis. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.														
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_A +		Retrieves the coefficient *A* of the line equation, for the marker occurrence. The line equation is of the general form, $Ax + By + C = 0$. If retrieved from an edge marker (measurement marker buffer), M_LINE_A returns the coefficient *A* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *A* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_A returns the coefficient *A* of the line joining the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_B +	Retrieves the coefficient *B* of the line equation, for the marker occurrence. The line equation is of the general form, *Ax* + *By* + *C* = 0. If retrieved from an edge marker (measurement marker buffer), M_LINE_B returns the coefficient *B* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *B* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_B returns the coefficient *B* of the line joining the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_C +	Retrieves the coefficient *C* of the line equation, for the marker occurrence. The line equation is of the general form, *Ax* + *By* + *C* = 0. If retrieved from an edge marker (measurement marker buffer), M_LINE_C returns the coefficient *C* of the line measured for the edge. For a stripe marker, this line follows the center of the stripe, and is calculated as the mean of the lines following its edges. If the result is requested for a specific edge of the stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, the coefficient *C* of the line equation of the first or second edge, respectively, is returned. If retrieved from a measurement result buffer, M_LINE_C returns the coefficient *C* of the line joining the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from any type of marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type marker (measurement marker buffer)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																

M_POSITION +	Retrieves the X- and Y-coordinates of the position, for the marker occurrence. For point markers, the position refers to the X- and Y-coordinates specified using MmeasSetMarker() with M_POSITION. For edge markers, the position refers to the X- and Y-coordinates of the edge's maximum edgevalue (highest edge peak) along the center of the search region. For stripe markers, the position refers to the X- and Y-coordinates at the center of a theoretical line between the position (maximum edgevalue) of the stripe's two outermost edges. If you use M_EDGE_FIRST or M_EDGE_SECOND, you will get the X- and Y-coordinates of that edge's maximum edgevalue (highest edge peak). For circle markers, the position refers to the X- and Y-coordinates at the circle's center.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALID_FLAG +	Retrieves whether the marker was found, or that the coordinates of a point marker (MmeasAllocMarker() with M_POINT) are set to a valid position.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the marker was not found.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the marker was found.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from an edge, stripe, or circle marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an edge, stripe, or circle marker (measurement marker buffer)																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																

M_FIT_ERROR_MAX +	Retrieves the maximum distance from a subedge to the fitted line equation (for edge or stripe markers) or to the fitted circle (for circle markers), for the marker occurrence. This distance is called the maximum fit error. If you did not specify subregions (for edges or stripes), M_FIT_ERROR_MAX returns 0. Note that circles always have subregions.															
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the maximum fit error. For a stripe marker, this is the maximum fit error for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the maximum fit error for the second edge of a stripe. For edge and circle markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_OUTLIERS +	Retrieves the number of subedges considered outliers, for the marker occurrence. Outliers are subedges within the search region that do not contribute to marker's fit operation since they are considered to be too far away from the general distribution of subedges. The number of outliers corresponds to the number of fitted subedges subtracted from the number of subregions. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of outliers for the edge or circle. For a stripe marker, this is the number of outliers for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of outliers for the second edge of a stripe. For edge and circle markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_WAS_CLIPPED +	Retrieves whether the search region was clipped. This value is only useful if you enabled clipping (MmeasSetMarker() with M_SEARCH_REGION_CLIPPING).	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the search region was not clipped.	c		e	f	g	a		i		j	k	l	m	n	b

		1	Specifies that the search region was clipped.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_EDGES_MARKER_INDEX +			Retrieves the index on which each subedge is located, for the marker occurrence. If a subedge is not found in a subregion, an arbitrary number is returned. Use M_SUB_EDGES_WEIGHT to determine if a subedge was actually found. This result is useful when retrieving subedge-type of results for a multiple-occurrence marker. For example, you can use the index to separate, by marker, the values obtained with M_SUB_EDGES_POSITION and M_SUB_EDGES_WEIGHT.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the index value. For a stripe marker, this is the index value of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns whether the result refers to the first or second edge of the stripe. For edge and circle markers, this parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EDGE_FIRST	Specifies that the edge is the first edge in the stripe.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EDGE_SECOND	Specifies that the edge is the second edge in the stripe.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_EDGES_POSITION +			Retrieves the X- and Y-coordinates of the subedges, for the marker occurrence. If a subedge is not found in a subregion, the X- and Y- positions will be arbitrary numbers. Use M_SUB_EDGES_WEIGHT to determine whether a subedge was found. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUB_EDGES_WEIGHT +	Retrieves the weight of the subedges, for the marker occurrence. The weight indicates whether a subedge was found in the subregion. Found subedges are used to calculate the fit, angle, and line equation values. Use M_SUB_EDGES_MARKER_INDEX to establish the occurrence to which the subedge belongs.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the subedge was not found.	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies that the subedge was found.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_NUMBER_USED +	Retrieves the number of subregions used. Note that if you did not specify multiple subregions (MmeasSetMarker() with M_SUB_REGIONS_NUMBER), Aurora Imaging Library will internally use three to perform certain measurements.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of subregions.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from an edge or stripe marker (measurement marker buffer), ResultType can be set to one of the values specified in the table below, unless otherwise specified.

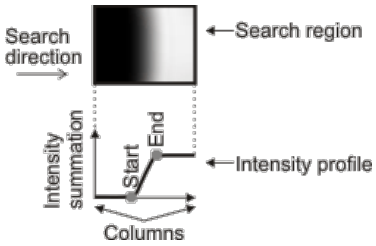
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

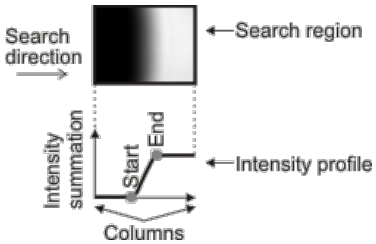
For an edge or stripe marker (measurement marker buffer)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstResultArrayPtr - Possible values returned																
SecondResultArrayPtr - Possible values returned																

M_BOX_ANGLE_FOUND +	Retrieves the angle found for the box search region when the angle is internally determined by Aurora Imaging Library. In this case, you must have specified M_ANY as the angle of the box search region (MmeasSetMarker() with M_BOX_ANGLE) or enabled an angular search (M_BOX_ANGLE_MODE set to M_ENABLE). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.														
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_BOTTOM_LEFT +	Retrieves the coordinates of the bottom-left corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_BOTTOM_RIGHT +	Retrieves the coordinates of the bottom-right corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b

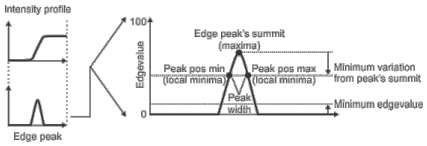
M_BOX_CORNER_TOP_LEFT +	Retrieves the coordinates of the top-left corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CORNER_TOP_RIGHT +	Retrieves the coordinates of the top-right corner of the marker's box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the result will take that rotation into account.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position on the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_EDGEVALUES +	Retrieves the edgevalues of the marker's box search region. Aurora Imaging Library establishes edgevalues by applying a first derivative filter to the search region's intensity profile and normalizing the output according to the number of pixels and the maximum pixel value possible. To establish the intensity profile, Aurora Imaging Library projects the pixels bounded by the two-dimensional search region (or each subregion) into a one-dimensional pixel intensity summation, which is performed vertically or horizontally, depending on the search region's origin and search direction.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns the edgevalues.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

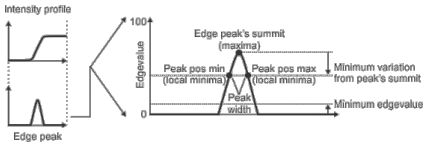
M_BOX_EDGEVALUES_NUMBER +	Retrieves the number of edgevalues that M_BOX_EDGEVALUES returns. This value corresponds to the number of pixels along the width or height of the box search region, depending on the search region's origin and search direction.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the number of edgevalues.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_FROM_BOX_ORIGIN +	Retrieves the position, as a distance value relative to the origin of the box search region, for the marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position as a distance value. For a stripe marker, this is the distance to the first edge of the stripe.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the position as the distance to the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_CONTRAST +	Retrieves the grayscale difference between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established, for the marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the contrast. For a stripe marker, this is the contrast of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the contrast of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_END +	Retrieves the X- and Y-coordinates of the end of the edge (that is, the end of the intensity transition from which the edge is established), for the marker occurrence. For a stripe marker, the coordinates representing the average value of end positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. In the intensity profile calculated for a search region, M_EDGE_START represents the first significant slope used to establish the edge, while M_EDGE_END represents the first significant slope in the opposite direction used to establish the edge. Depending on the search region's origin and search direction, the edge's start and end can be on either side of the edge.  To retrieve the minimum and maximum position of the edge peak of the marker, use M_EDGEVALUE_PEAK_POS_MIN and M_EDGEVALUE_PEAK_POS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the end of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the end of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_INSIDE +	Retrieves the number of edges located between the two exterior edges of a stripe, for the marker occurrence. This result only applies to stripes.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_START +	Retrieves the X- and Y-coordinates of the start of the edge (that is, the start of the intensity transition from which the edge is established), for the marker occurrence. For a stripe marker, the coordinates representing the average value of start positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. In the intensity profile calculated for a search region, M_EDGE_START represents the first significant slope used to establish the edge, while M_EDGE_END represents the first significant slope in the opposite direction used to establish the edge. Depending on the search region's origin and search direction, the edge's start and end can be on either side of the edge.  To retrieve the minimum and maximum position of the edge peak of the marker, use M_EDGEVALUE_PEAK_POS_MIN and M_EDGEVALUE_PEAK_POS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the start of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the start of the edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_STRENGTH +	Retrieves the greatest edgevalue of the edge, for the marker occurrence. The greatest edgevalue of an edge is considered to be its edge strength.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the edge strength. For a stripe marker, this is the edge strength for the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the strength for the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGE_WIDTH +	Retrieves the distance between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the edge is established, for the marker occurrence. To retrieve the width of the edge established by the edge peak of the marker, use M_EDGEVALUE_PEAK_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the edge width. For a stripe marker, this is the edge width of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the edge width of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_CONTRAST +	Retrieves the grayscale difference of the intensity transition between the first zero edgevalues on both sides of the established edge peak (before M_EDGEVALUE_PEAK_POS_MIN, and after M_EDGEVALUE_PEAK_POS_MAX), for the marker occurrence. Note that some images might never have a zero edgevalue, in which case you should use M_EDGE_CONTRAST for a more meaningful result.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the contrast. For a stripe marker, this is the contrast of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the contrast of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_PEAK_POS_MAX +	Retrieves the X- and Y-coordinates of the maximum position of the edge peak (that is, the maximum edgevalue along the first derivative representation of the intensity profile), for the marker occurrence. For a stripe marker, the coordinates representing the average value of maximum positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. To establish the maximum and minimum positions of the edge peak, Aurora Imaging Library considers the portion of the peak that contains the maximum edgevalue, and that satisfies the M_EDGEVALUE_MIN and M_EDGEVALUE_VAR_MIN constraints set using MmeasSetMarker(). That is, the peak must dip, on both sides, below either the minimum edgevalue threshold (M_EDGEVALUE_MIN) or the local edgevalue threshold established by subtracting M_EDGEVALUE_VAR_MIN from the peak's summit, whichever is highest.  Depending on the search region's origin and search direction, the maximum and minimum position can be on either side of the edge peak's summit. Note that you can also retrieve different start and end positions of the edge using M_EDGE_START and M_EDGE_END.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the maximum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the maximum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_PEAK_POS_MIN +	Retrieves the X- and Y-coordinates of the minimum position of the edge peak (that is, the minimum edgevalue along the first derivative representation of the intensity profile), for the marker occurrence. For a stripe marker, the coordinates representing the average value of minimum positions is returned unless otherwise specified by the combination value M_EDGE_FIRST or M_EDGE_SECOND. To establish the minimum and maximum positions of the edge peak, Aurora Imaging Library considers the portion of the peak that contains the maximum edgevalue, and that satisfies the M_EDGEVALUE_MIN and M_EDGEVALUE_VAR_MIN constraints set using MmeasSetMarker(). That is, the peak must dip, on both sides, below either the minimum edgevalue threshold (M_EDGEVALUE_MIN)or the local edgevalue threshold established by subtracting M_EDGEVALUE_VAR_MIN from the peak's summit, whichever is highest.  Depending on the search region's origin and search direction, the maximum and minimum position can be on either side of the edge peak's summit. Note that you can also retrieve different start and end positions of the edge using M_EDGE_START and M_EDGE_END.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the minimum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the minimum position of the edge peak.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_WIDTH +	Retrieves the distance between the minimum (M_EDGEVALUE_PEAK_POS_MIN) and maximum (M_EDGEVALUE_PEAK_POS_MAX) position of the edge peak (that is, the minimum and maximum positions of the edgevalue along the first derivative representation of the intensity profile), for the marker occurrence. To retrieve the width of the edge instead of the width of the edge peak, use M_EDGE_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the edge width. For a stripe marker, this is the edge width of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b

	SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the edge width of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +		Retrieves the length of the side of the search region perpendicular to the search direction. This is useful to retrieve the number of pixels that were projected into a single value of the intensity profile without having to find M_ORIENTATION first.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_END_POINT_FIRST +		Retrieves the X- and Y-coordinates of the first intersection point, for the marker occurrence, between the edge's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you request this result for a specific edge of a stripe, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, M_LINE_END_POINT_FIRST will return the first intersection point of the line following the specified edge and the box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the intersection will take that rotation into account. For vertical edges, the first intersection point is typically at the top of the box search region, while for horizontal edges the first intersection point is typically on the right of the box search region. Note that the intersection point that is considered to be the first or second (M_LINE_END_POINT_SECOND) can change depending on the edge's angle and whether the box search region is rotated.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_END_POINT_SECOND +	Retrieves the X- and Y-coordinates of the second intersection point, for the marker occurrence, between the edge's mean line and the box search region. For a stripe marker, the mean line is the mean of the lines following its two edges. If you request a result for a specific edge of a stripe marker, using the combination value M_EDGE_FIRST or M_EDGE_SECOND, M_LINE_END_POINT_SECOND returns the second intersection point of the line following the specified edge and the box search region. If the box search region was rotated using MmeasSetMarker() with M_BOX_ANGLE or from an angular search, the intersection will take that rotation into account. For vertical edges, the second intersection point is typically at the bottom of the box search region, while for horizontal edges the second intersection point is typically on the left of the box search region. Note that the intersection point that is considered to be the first (M_LINE_END_POINT_FIRST) or second can change depending on the edge's angle and whether the box search region is rotated.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the intersection point.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIENTATION +	Retrieves the orientation of the marker.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL	Specifies a horizontal orientation.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL	Specifies a vertical orientation.	c		e	f	g	a		i		j	k	l	m	n	b
SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_POLARITY +	Retrieves the polarity, for the marker occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the polarity of the edge. For a stripe marker, this is the polarity of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b

		M_NEGATIVE	Specifies that the polarity of the edge is negative.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITIVE	Specifies that the polarity of the edge is positive.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns the polarity of the second edge of a stripe. For edge markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NEGATIVE	Specifies that the polarity of the second edge is negative.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITIVE	Specifies that the polarity of the second edge is positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +			Retrieves the score, for the marker occurrence. This score represents how confident you should be that Aurora Imaging Library found what you expected. For a stripe marker, the score is the average of the score of both edges.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr		Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_TOTAL +			Retrieves the marker's final score. This score represents how confident you should be that Aurora Imaging Library found what you expected. Aurora Imaging Library establishes the marker's final score using all marker occurrences and score characteristics. For a stripe marker, the score is the average of the score of both edges.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr		Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING +			Retrieves the distance between the marker occurrence and the next.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr		Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE_WIDTH +			Retrieves the width of the stripe, for the marker occurrence. The stripe's width refers to the distance between the position of the maximum edgevalue of the stripe's two outermost edges. This result is only available for stripes.	c		e	f	g	a		i		j	k	l	m	n	b

	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_SUB_EDGES_MARKER_INDEX; M_SUB_EDGES_POSITION; M_SUB_EDGES_WEIGHT; M_BOX_EDGEVALUES.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstResultArrayPtr - Possible values returned																
	SecondResultArrayPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For an edge or stripe marker (measurement marker buffer) or a measurement result buffer; and for the following values: M_POSITION; M_FIT_ERROR_MAX; M_NUMBER_OF_OUTLIERS; M_SUB_EDGES_MARKER_INDEX; M_SUB_EDGES_POSITION; M_SUB_EDGES_WEIGHT; M_DISTANCE_FROM_BOX_ORIGIN; M_EDGE_CONTRAST; M_EDGE_END; M_EDGE_START; M_EDGE_STRENGTH; M_EDGE_WIDTH; M_EDGEVALUE_PEAK_CONTRAST; M_EDGEVALUE_PEAK_POS_MAX; M_EDGEVALUE_PEAK_POS_MIN; M_EDGEVALUE_PEAK_WIDTH; M_LENGTH; M_LINE_END_POINT_FIRST; M_LINE_END_POINT_SECOND; M_POLARITY; M_SPACING.

You can add one of the following values to the above-mentioned values to specify the edge for which to retrieve the result (stripe markers only).

Note that when retrieving non-positional results, if you explicitly request the result for a specific edge of a stripe, using M_EDGE_FIRST or M_EDGE_SECOND, you must retrieve the result for that edge with FirstResultArrayPtr. In this case, you must set SecondResultArrayPtr to **M_NULL**. For positional results, you can set FirstResultArrayPtr or SecondResultArrayPtr to **M_NULL** if you only want to retrieve the X- or Y-coordinates, respectively. You must always pass at least one pointer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the edge for which to retrieve the result (stripe markers only)																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_FIRST	Retrieves the specified result for the first outermost edge of a stripe.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_SECOND	Retrieves specified result for the second outermost edge of a stripe.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from a circle marker (measurement marker buffer), ResultType can be set to the value specified in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a circle marker (measurement marker buffer)																		
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstResultArrayPtr - Possible values returned																	
	SecondResultArrayPtr - Possible values returned																	
M_RADIUS +		Retrieves the circle's radius.																
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results from a measurement result buffer, the ResultType parameter can be set to one of the following values. If you are using two multiple-occurrence markers that have a different number of occurrences, Aurora Imaging Library uses the fewest number of occurrences to calculate results (MmeasCalculate()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement result buffer																		
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstResultArrayPtr - Possible values returned																	
	SecondResultArrayPtr - Possible values returned																	
M_DISTANCE +		Retrieves the distance between the occurrence of the two markers.																
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_X +		Retrieves the distance on the X-axis between the occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_Y +		Retrieves the distance on the Y-axis between the occurrence of the two markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For any type of marker (measurement marker buffer) or a measurement result buffer
- For an edge or stripe marker (measurement marker buffer) or a measurement result buffer
- For any type marker (measurement marker buffer)

- For an edge, stripe, or circle marker (measurement marker buffer)
- For an edge or stripe marker (measurement marker buffer)
- For a circle marker (measurement marker buffer)
- For a measurement result buffer

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the required data type																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstResultArrayPtr - Possible values returned																
	SecondResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE		Casts the requested results to an *AIL_DOUBLE*.															
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT		Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstResultArrayPtr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

	SecondResultArrayPtr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasGetScore

Synopsis

Retrieves information about the specified score characteristic.

Syntax

```
void MmeasGetScore(
    AIL_ID MarkerId,          //in
    AIL_INT64 Characteristic, //in
    AIL_INT Index,            //in
    AIL_DOUBLE *Param1Ptr,    //out
    AIL_DOUBLE *Param2Ptr,    //out
    AIL_DOUBLE *Param3Ptr,    //out
    AIL_DOUBLE *Param4Ptr,    //out
    AIL_DOUBLE *Param5Ptr,    //out
    AIL_INT *InputUnitsPtr,   //out
    AIL_INT64 ControlFlag     //in
)
```

Description

This function retrieves information about the specified score characteristic of an edge, stripe, or circle marker. When using this function with M_RESULT, you will get the results of the score characteristic that Aurora Imaging Library established in the target image after your last call to MmeasFindMarker(), as well as the value of the characteristic established by the find operation. When using this function with M_MARKER, you will get the values that you specified for the score characteristic's settings with your last call to MmeasSetScore(), as well as the value of the character specified in the marker. Before using this function with M_RESULT, you must first call MmeasFindMarker().

The information that you can retrieve for each score characteristic is based on its settings in MmeasSetScore(). If you try to get the results (M_RESULT) of a score characteristic that you did not set using MmeasSetScore(), you will receive an error. However, if you inquire (M_MARKER) about a score characteristic that you did not set, you will receive the default (initial) values of that characteristic, as indicated in MmeasSetScore().

If you have associated the target image with a camera calibration context (MmeasFindMarker()), you can specify that certain values be interpreted in world units. To do so, you must have used MmeasSetScore() with M_WORLD. Note that if you used M_WORLD but didn't use a calibrated target image, the function will generate an error.

Parameters

MarkerId

Specifies the identifier of the measurement marker buffer.

Characteristic

Specifies the score characteristic about which to get information.

For retrieving information about a score characteristic for an edge, stripe, or circle marker, the Characteristic parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a score characteristic for an edge, stripe, or circle marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_EDGE_CONTRAST_SCORE +	Retrieves information about the grayscale difference in intensity between the start (M_EDGE_START) and end (M_EDGE_END) of the intensity transition from which the marker's edges are established. You must specify a combination value from each of the following tables: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic • To specify the edge of the stripe for which to return results (stripe markers only)	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_CONTRAST_SCORE +	Retrieves information about the grayscale difference of the intensity transition between the first zero edgevalues on both sides of the established edge peak (before M_EDGEVALUE_PEAK_POS_MIN, and after M_EDGEVALUE_PEAK_POS_MAX). Note that some images might never have a zero edgevalue, in which case you should use M_EDGE_CONTRAST_SCORE for a more meaningful result. You must specify a combination value from each of the following tables: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic • To specify the edge of the stripe for which to return results (stripe markers only)	c		e	f	g	a		i		j	k	l	m	n	b
M_STRENGTH_SCORE +	Retrieves information about the greatest edgevalue (strength) of the marker. You must specify a combination value from each of the following tables: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic • To specify the edge of the stripe for which to return results (stripe markers only)	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For retrieving information about a score characteristic for an edge, stripe, or circle marker.

You must add one of the following values to the above-mentioned values to specify the edge of the stripe for which to return results (stripe markers only).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the stripe's edge																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_FIRST	Retrieves results for the first outermost edge of a stripe. This must be combined with M_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_SECOND	Retrieves results for the second outermost edge of a stripe. This must be combined with M_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

For retrieving information about a score characteristic for an edge or stripe marker, the Characteristic parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a score characteristic for an edge or stripe marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISTANCE_FROM_BOX_ORIGIN_SCORE +	Retrieves information about the distance between the position of the box search region's origin and the position of the marker. You must specify a combination value from the following table: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING_SCORE +	Retrieves information about the typical distance (spacing) between consecutive edge or stripe occurrences of a multiple-occurrence marker. You must specify a combination value from the following table: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic	c		e	f	g	a		i		j	k	l	m	n	b

For retrieving information about a score characteristic for a stripe marker, the Characteristic parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a score characteristic for a stripe marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_EDGE_INSIDE_SCORE +	Retrieves information about the number of edges inside a stripe marker. You must specify a combination value from the following table: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE_WIDTH_SCORE +	Retrieves information about the distance (width) between the stripe's two outermost edges. You must specify a combination value from the following table: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic	c		e	f	g	a		i		j	k	l	m	n	b

For retrieving information about a score characteristic for a circle marker, the Characteristic parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving information about a score characteristic for a circle marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RADIUS_SCORE +	Retrieves information about the linear distance between the circle's center and its circumference (radius). You must specify a combination value from the following table: • To specify whether to retrieve information about the results of, or the values set for, the specified score characteristic	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving information about a score characteristic for an edge, stripe, or circle marker
- For retrieving information about a score characteristic for an edge or stripe marker
- For retrieving information about a score characteristic for a stripe marker
- For retrieving information about a score characteristic for a circle marker

the values listed in all the values group tables.

You must add one of the following values to the above-mentioned values to specify whether to retrieve information about the results of, or the values set for, the specified score characteristic.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to retrieve information about the results of, or the values set for, the specified score characteristic																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MARKER	Retrieves the values that you specified for the score characteristic's settings with your last call to MmeasSetScore().	c		e	f	g	a		i		j	k	l	m	n	b

M_RESULT	Retrieves the results of the score characteristic after your last call to MmeasFindMarker(). Aurora Imaging Library returns values using the input units specified using MmeasSetScore().	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the index of the occurrence of a multiple-occurrence marker. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the occurrence of a multiple-occurrence marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the index information is not applicable. Use this value when using M_MARKER.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the occurrence of a multiple-occurrence marker (MarkerId), when using M_RESULT. If you are using M_RESULT, but did not specify a multiple-occurrence marker, the index must be zero.	c		e	f	g	a		i		j	k	l	m	n	b

Param1Ptr

Specifies the address of the variable in which to write either the characteristic's resulting value (when using M_RESULT) or the characteristic's minimum acceptable limit, as indicated using MmeasSetScore() with the Min parameter (when using M_MARKER).

When using M_RESULT, the characteristic's resulting value (as measured in the target image using MmeasFindMarker()) is the same as the corresponding characteristic result retrieved using MmeasGetResult() and MmeasGetResultSingle(), regardless of whether you specify a score characteristic using MmeasSetScore(). This correspondence is listed in the following table:

MmeasGetScore() and Param1Ptr	MmeasGetResult() and MmeasGetResultSingle()
M_EDGE_CONTRAST_SCORE	M_EDGE_CONTRAST
M_EDGEVALUE_PEAK_CONTRAST_SCORE	M_EDGEVALUE_PEAK_CONTRAST
M_STRENGTH_SCORE	M_EDGE_STRENGTH
M_DISTANCE_FROM_BOX_ORIGIN_SCORE	M_DISTANCE_FROM_BOX_ORIGIN
M_SPACING_SCORE	M_SPACING
M_RADIUS_SCORE	M_RADIUS

M_EDGE_INSIDE_SCORE	M_EDGE_INSIDE
M_STRIPE_WIDTH_SCORE	M_STRIPE_WIDTH

Param2Ptr

Specifies the address of the variable in which to write either the characteristic's resulting score (when using M_RESULT) or the characteristic's lowest acceptable limit, as indicated using MmeasSetScore() with the Low parameter (when using M_MARKER).

Param3Ptr

Specifies the address of the variable in which to write either the greatest value that Aurora Imaging Library considers theoretically possible for the characteristic (when using M_RESULT) or the characteristic's highest acceptable limit, as indicated using MmeasSetScore() with the High parameter (when using M_MARKER).

Param4Ptr

Specifies the address of the variable in which to write the characteristic's maximum acceptable limit, as indicated using MmeasSetScore() with the Max parameter (when using M_MARKER); otherwise, you must specify **M_NULL**, indicating that Aurora Imaging Library does not return any information to this parameter (when using M_RESULT).

Param5Ptr

Specifies the address of the variable in which to write the characteristic's score-offset, as indicated using MmeasSetScore() with the ScoreOffset parameter (when using M_MARKER); otherwise, you must specify **M_NULL**, indicating that Aurora Imaging Library does not return any information to this parameter (when using M_RESULT).

InputUnitsPtr

Specifies the address of the variable in which to write the characteristic's input units, as indicated using MmeasSetScore() with the InputUnits parameter (when using M_MARKER); otherwise, you must specify **M_NULL**, indicating that Aurora Imaging Library does not return any information to this parameter (when using M_RESULT).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the input units																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_PIXEL, if the score characteristic's settings are affected by input units. Otherwise, M_DEFAULT specifies that the input unit information should be ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that the values are retrieved in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the values are retrieved in world units, with respect to the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasInquire

Synopsis

Inquire about a measurement context, marker, or result buffer.

Syntax

```
AIL_INT MmeasInquire(
    AIL_ID MeasId,         //in
    AIL_INT64 InquireType, //in
    void *FirstUserVarPtr, //out
    void *SecondUserVarPtr //out
)
```

Description

This function inquires information about an expected marker characteristic, context setting, or measurement result buffer setting. To specify a marker characteristic, use MmeasSetMarker(); to specify settings for a measurement context or measurement result buffer, use MmeasControl().

To retrieve the results of a find marker or calculate measurement operation (MmeasFindMarker() or MmeasCalculate()), use MmeasGetResult() or MmeasGetResultSingle().

If the inquired setting is set to M_DEFAULT (for example, in MmeasControl()), **MmeasInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

MeasId

Specifies the identifier of the marker, measurement context, or measurement result buffer about which to inquire information.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the measurement object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default measurement context of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
Measurement context ID	Specifies a measurement context.	c		e	f	g	a		i		j	k	l	m	n	b
Measurement marker ID	Specifies a measurement marker.	c		e	f	g	a		i		j	k	l	m	n	b
Measurement result buffer ID	Specifies a measurement result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

For certain inquire types, two values can be returned for a stripe marker (one for each of the edges). In these cases, the value for the first edge is returned to FirstUserVarPtr and that of the second edge to SecondUserVarPtr. For an edge or circle marker, all results are returned to FirstUserVarPtr. Inquire types which return a position always return the X-coordinate to FirstUserVarPtr and the Y-coordinate to SecondUserVarPtr.

See the Parameter associations section for possible values that can be specified.

FirstUserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MmeasInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

SecondUserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64
- M_NULL

Specifies the address in which to write the requested information. If only one value is to be returned, it will be returned to FirstUserVarPtr and SecondUserVarPtr should be set to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **FirstUserVarPtr** and **SecondUserVarPtr** parameters.

- For point, edge, stripe, and circle markers, and for a measurement result buffer
- For point, edge, stripe, and circle markers
- For point, edge, and stripe markers
- For edge, stripe, and circle markers
- For edge and stripe markers
- For point markers
- For stripe markers
- For circle markers
- For a measurement context buffer
- For a measurement result buffer
- For inquiring the system on which the measurement context, marker, or result buffer was allocated

For point, edge, and stripe markers, and for measurement result buffers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For point, edge, stripe, and circle markers, and for a measurement result buffer																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstUserVarPtr - Possible values returned																

SecondUserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For point, edge, stripe, and circle markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For point, edge, stripe, and circle markers																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_MARKER_REFERENCE +	Inquires the offset with which to establish the marker's reference position, relative to the marker's actual position. SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-offset value, in the units of the input coordinate system specified using M_MARKER_REFERENCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-offset value, in the units of the input coordinate system specified using M_MARKER_REFERENCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_MARKER_REFERENCE_INPUT_UNITS +	Inquires the units with which to interpret the M_MARKER_REFERENCE inquire type. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_MARKER_TYPE +	Inquires the type of marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE	Specifies a circle marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE	Specifies an edge marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT	Specifies a point marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRIPE	Specifies a stripe marker buffer.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For point, edge, and stripe markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For point, edge, and stripe markers																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_NUMBER +	Inquires the number of edges or stripes to locate, or point markers defined. SET															

FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies that the search should locate all edges or stripes in the box search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of edges or stripes to locate, or point markers to define.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For edge, stripe, and circle markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For edge, stripe, and circle markers																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_EDGEVALUE_MIN +	Inquires the minimum edgevalue required for it to be considered an occurrence of the marker. SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum edgevalue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_VAR_MIN +	Inquires the minimum prominence required for an edge peak, for it to be considered an occurrence of the marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum prominence. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS +	Inquires the degree of smoothness (strength of denoising) applied to the internal projection buffer of the search region during the edge extraction. SET	c		e	f	g	a		i		j	k	l	m	n	b

FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the smoothness value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE +	Inquires the type of the first derivative filter with which to perform the edge extraction. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_EULER	Specifies an Euler FIR filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREWITT	Specifies a Prewitt FIR filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SHEN	Specifies a Shen-Castan Infinite Support Exponential filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_ASSOCIATION_DISTANCE +	Inquires the maximum distance between an edge (either straight or circular) and its associated subedges. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the distance, in the units of the input coordinate system specified using M_MAX_ASSOCIATION_DISTANCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_ASSOCIATION_DISTANCE_INPUT_UNITS +	Inquires the units with which to interpret the M_MAX_ASSOCIATION_DISTANCE inquire type. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_POLARITY +	Inquires the polarity of the marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the polarity of the edge. For stripe markers, this is the polarity of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies that the polarity of the edge is negative (falling edge). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies that the polarity of the edge is positive (rising edge). INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the polarity of the second edge of a stripe marker. For edge and circle markers, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the polarity is not considered.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies that the polarity of the second edge is negative (falling edge).	c		e	f	g	a		i		j	k	l	m	n	b
M_OPPOSITE	Specifies that the polarity of the second edge of a stripe marker is the opposite of that of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies that the polarity of the second edge is positive (rising edge).	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the polarity of the second edge of a stripe marker is the same as that of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_ANGLE_INTERPOLATION_MODE +	Inquires the type of interpolation to use. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC	Specifies bicubic interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING +		Inquires whether Aurora Imaging Library can automatically clip the search region (box or ring) when it falls outside the image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DISABLE	Specifies that the search region cannot be automatically clipped. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAXIMIZE_AREA	Specifies that, if the search region falls outside the image, Aurora Imaging Library can automatically clip the region according to the best valid area in which to search, and the marker can still be found. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_MIN_AREA +		Inquires the minimum area requirement for the clipped region (area that Aurora Imaging Library searches for a marker) that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the minimum area required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_MIN_HEIGHT +		Inquires the minimum height requirement for the clipped region (area that Aurora Imaging Library searches for a marker) that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the minimum height required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_REGION_CLIPPING_MIN_WIDTH +	Inquires the minimum width requirement for the clipped region (area that Aurora Imaging Library searches for a marker) that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum width required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_PRESERVE_CENTER +	Inquires the center requirement for the clipped region (area that Aurora Imaging Library searches for a marker) that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALONG_HEIGHT	Specifies that the center, along the height of the defined search region, must be preserved in the clipped region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALONG_WIDTH	Specifies that the center, along the width of the defined search region, must be preserved in the clipped region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library will determine how the center of the defined search region will be preserved in the clipped region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that the clipped region need not preserve the center of the defined search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_INPUT_UNITS +	Inquires the units with which to interpret the M_BOX_CENTER, M_BOX_ORIGIN, M_BOX_SIZE, M_BOX_ANGLE, M_BOX_ANGLE_ACCURACY, M_BOX_ANGLE_TOLERANCE, M_BOX_ANGLE_DELTA_POS, M_BOX_ANGLE_DELTA_NEG, M_RING_CENTER, M_RING_RADII, and M_SUB_REGIONS_CHORD_ANGLE inquire types. SET	c		e	f	g	a		i		j	k	l	m	n	b

	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PIXEL	Specifies to interpret the values in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		M_WORLD	Specifies to interpret the values in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_NUMBER +		Inquires the number of sections (subregions) in which to divide the search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		1 <= *Value* < 8	Specifies a number of sections (subregions) that can only be used for edge and stripe markers.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 8	Specifies a number of sections (subregions) that can be used for edge, stripe, and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBPIXEL_MODE +		Inquires how Aurora Imaging Library calculates subpixel positions. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GLOBAL	Specifies that Aurora Imaging Library calculates subpixel positions based on the entire region within which the edge is found. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOCAL	Specifies that Aurora Imaging Library calculates subpixel positions based on the immediate neighborhood within which the edge's maximum edgevalue is found. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For edge and stripe markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For edge and stripe markers																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_BOX_ANGLE +	Inquires the angle of the box search region. An angle interpreted according to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the content of the box search region is analyzed to automatically determine the angle of the marker.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 360	Specifies the angle of the box search region, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_ACCURACY +	Inquires the accuracy of the angular search. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies the angle of accuracy to be equal to the angle of tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 180.0	Specifies the accuracy of the angular search, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_DELTA_NEG +	Inquires the negative range of angles within which to search for the marker. SET	c		e	f	g	a		i		j	k	l	m	n	b

FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default negative range of angles, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 360.0	Specifies the negative range of angles, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_DELTA_POS +	Inquires the positive range of angles within which to search for the marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default positive range of angles, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 360.0	Specifies the positive range of angles, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_MODE +	Inquires whether a angular search is enabled. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the search should only be performed at the angle specified by M_BOX_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that multiple-angle search is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_REFERENCE +	Inquires the center of rotation used when performing a search at an angle. SET	c		e	f	g	a		i		j	k	l	m	n	b

FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CENTER	Specifies the center of the box search region as the center of rotation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ORIGIN	Specifies the top-left corner of the box search region as the center of rotation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_TOLERANCE +	Inquires the rotation tolerance of the marker. This is the full range of degrees within which a marker can be rotated from a box search region that is at a specific angle and still be found. This determines the step angle used for an angular search. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0.1 <= *Value* <= 360.0	Specifies the rotation tolerance, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CENTER +	Inquires the coordinates of the center of the box search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the center of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default X-coordinate of the center of the box search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the center of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the center of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default Y-coordinate of the center of the box search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the Y-coordinate of the center of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ORIGIN +	Inquires the coordinates of the origin of the box search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the origin of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the origin of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_SIZE +	Inquires the width and height of the box search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the width of the box search region within the target image.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the width of the entire target image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the box search region within the target image, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the height of the box search region within the target image.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the height of the entire target image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the box search region within the target image, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PROFILE_SCALE_OFFSET +	Inquires the scale and offset used with M_DRAW_..._PROFILE. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b

		M_AUTO_SCALE_PROFILE	Specifies to automatically compute the greatest scale factor that will fit the entire drawing within the destination buffer (or within the area that corresponds to the search region, if M_DRAW_IN_BOX is combined when drawing) after the offset has been applied.	c		e	f	g	a		i		j	k	l	m	n	b
		Value > 0.0	Specifies the scale multiplier.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_AUTO_OFFSET_PROFILE	Specifies to automatically compute the offset such that the maximum and minimum of the drawing are touching the ends of the destination buffer (or the area that corresponds to the search region, if M_DRAW_IN_BOX is combined when drawing). INFO	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the offset from the middle of the height of the destination buffer or the area corresponding to the search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MIN +			Inquires the minimum number of edges or stripes to locate. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the minimum number of edges or stripes to locate.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIENTATION +			Inquires the orientation of the marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the orientation of the marker is unknown; both horizontal and vertical orientations are searched.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HORIZONTAL	Specifies that the marker has a horizontal orientation.	c		e	f	g	a		i		j	k	l	m	n	b

M_VERTICAL	Specifies that the marker has a vertical orientation.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_OFFSET +	Inquires the offset of the subregions, from the center point of each of their sections. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
-100 <= *Value* <= 100	Specifies the offset of the subregions.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_SIZE +	Inquires the size of the subregions, as a percentage of their section. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the size of the subregions, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For point markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For point markers																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_MULTIPLE_POINT_ANGLE +	Inquires the angle of the theoretical line along which subsequent points are placed, for a multiple point marker. An angle interpreted according to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. SET															

FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 360	Specifies the angle for a multiple point marker, in degrees, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_INPUT_UNITS +	Inquires the units with which to interpret the M_POSITION, M_SPACING, and M_MULTIPLE_POINT_ANGLE inquire types. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION +	Inquires the coordinates of a point marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the point marker.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the position is invalid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the point marker, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the point marker.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies that the position is invalid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the point marker, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING +	Inquires the spacing (distance) between consecutive points used as a multiple-occurrence marker. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b

		M_ANY	Specifies that the spacing between points has not been set. M_ANY is only valid if M_NUMBER is set to 1. Otherwise, M_ANY is an invalid value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the spacing, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For stripe markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For stripe markers																		
InquireType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstUserVarPtr - Possible values returned																	
	SecondUserVarPtr - Possible values returned																	
M_INCLUSION_POINT +			Inquires the position of the inclusion point. SET															
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns the inclusion point's X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies no inclusion point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the inclusion point's X-coordinate, in the units of the input coordinate system specified using M_INCLUSION_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns the inclusion point's Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies no inclusion point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the inclusion point's Y-coordinate, in the units of the input coordinate system specified using M_INCLUSION_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUSION_POINT_INPUT_UNITS +			Inquires the units with which to interpret the M_INCLUSION_POINT inquire type. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUSION_POINT_INSIDE_STRIPE +			Inquires whether the inclusion point must be inside the stripe. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that M_INCLUSION_POINT is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NO	Specifies that only stripes that have the inclusion point falling outside its two outer edges can be found.	c		e	f	g	a		i		j	k	l	m	n	b
		M_YES	Specifies that only stripes that have the inclusion point falling between its two outer edges can be found.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr		Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

For circle markers, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For circle markers																		
InquireType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstUserVarPtr - Possible values returned																	
	SecondUserVarPtr - Possible values returned																	
M_CIRCLE_ACCURACY +			Inquires the accuracy of the circle's fit operation. SET															
	FirstUserVarPtr		Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HIGH	Specifies that the circle's fit operation will be performed with high accuracy. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_LOW	Specifies that the circle's fit operation will be performed with low accuracy. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE_INSIDE_SEARCH_REGION +	Inquires whether the circle's fit operation can result in a circle that is outside the ring search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the circle's fit operation can result in a circle that is outside the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the circle's fit operation cannot result in a circle that is outside the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_RING_CENTER +	Inquires the coordinates of the center of the ring search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the X-coordinate of the center of the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default X-coordinate of the center of the ring search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the center of the ring search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the Y-coordinate of the center of the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default Y-coordinate of the center of the ring search region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the center of the ring search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_RING_RADII +	Inquires the inner and outer radius of the circle marker's ring-shaped region. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the inner radius.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies an inner radius of 0.0. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the inner radius, relative to the center of the ring search region, in the units specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the outer radius.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default outer radius. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > *Inner radius*	Specifies the outer radius, relative to the center of the ring search region, in the units specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_CHORD_ANGLE +	Inquires the angle with which to set the ring search region's outer ring segment (chord). An angle interpreted according to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup. SET	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 45.0	Specifies the angle, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

When performing an inquiry on a measurement context buffer, InquireType can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement context buffer																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	FirstUserVarPtr - Possible values returned																
	SecondUserVarPtr - Possible values returned																
M_PIXEL_ASPECT_RATIO +		Inquires the ratio of the width of the pixel to its height. SET															
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the pixel width to pixel height ratio.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO_INPUT +		Inquires how the Measurement module interprets specified measurement characteristics relative to the pixel aspect ratio. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CORRECTED	Specifies to correct measurement inputs with the specified aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NORMAL	Specifies to not correct measurement inputs (ratio of 1.0).	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO_OUTPUT +		Inquires how the Measurement module returns results relative to the pixel aspect ratio. SET	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CORRECTED	Specifies to correct measurement results with the specified aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NORMAL	Specifies to not correct measurement results (ratio of 1.0).	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

When performing an inquiry on a measurement result buffer, InquireType can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a measurement result buffer																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_RESULT_TYPE +	Inquires the type of result buffer allocated. SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE	Specifies a result buffer for an MmeasCalculate() operation.	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire the system on which the measurement context, marker, or result buffer was allocated, set the InquireType parameter to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the system on which the measurement context, marker, or result buffer was allocated																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the measurement context, marker, or result buffer has been allocated. SET SET SET															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For point, edge, stripe, and circle markers, and for a measurement result buffer
- For point, edge, stripe, and circle markers
- For point, edge, and stripe markers
- For edge, stripe, and circle markers
- For edge and stripe markers

- For point markers
- For stripe markers
- For circle markers
- For a measurement context buffer
- For a measurement result buffer
- For inquiring the system on which the measurement context, marker, or result buffer was allocated

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine the default value of an inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	FirstUserVarPtr - Possible values returned																
	SecondUserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type, regardless of the current value of the inquire type.															
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For point, edge, stripe, and circle markers, and for a measurement result buffer
- For point, edge, stripe, and circle markers
- For point, edge, and stripe markers
- For edge, stripe, and circle markers
- For edge and stripe markers
- For point markers
- For stripe markers
- For circle markers
- For a measurement context buffer
- For a measurement result buffer
- For inquiring the system on which the measurement context, marker, or result buffer was allocated

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType		Description															
	FirstUserVarPtr - Possible values returned																

SecondUserVarPtr - Possible values returned																
M_SUPPORTED	Inquires whether the specified inquire type is supported.															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE* Returns one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For point, edge, stripe, and circle markers, and for a measurement result buffer
- For point, edge, stripe, and circle markers
- For point, edge, and stripe markers
- For edge, stripe, and circle markers
- For edge and stripe markers
- For point markers
- For stripe markers
- For circle markers
- For a measurement context buffer
- For a measurement result buffer
- For inquiring the system on which the measurement context, marker, or result buffer was allocated

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstUserVarPtr - Possible values returned																
SecondUserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
FirstUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
FirstUserVarPtr	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
SecondUserVarPtr	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b

	FirstUserVarPtr	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstUserVarPtr	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	SecondUserVarPtr	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, typically returned to FirstUserVarPtr, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasRestoreMarker

Synopsis

Restore a marker from disk.

Syntax

```
AIL_ID MmeasRestoreMarker(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *MarkerIdPtr          //out
)
```

Description

This function restores a marker that was previously saved, using MmeasSaveMarker() or MmeasStream(), and returns a handle to it. It also restores all the marker characteristics that were in effect when the marker was saved.

When the restored marker is no longer required, release it using MmeasFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored marker's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the marker. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, marker files have an MRK extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the measurement marker.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion, and must be set to **M_DEFAULT**.

MarkerIdPtr

Specifies the address of the variable in which to write the marker identifier or specifies the data type that the function should use to return the marker identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement marker ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated measurement marker ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MEAS_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the measurement marker (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored marker. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the marker identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MEAS_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasSaveMarker

Synopsis

Save a marker to disk.

Syntax

```
void MmeasSaveMarker(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID MarkerId,             //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves an allocated marker to disk, including all of its current characteristics. The marker and its characteristics can later be restored, using MmeasRestoreMarker() or MmeasStream().

Parameters

FileName

Specifies the name and path of the file in which to save the marker. It is recommended that you use the MRK file extension for easier use with other Zebra Aurora software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For displaying results in an interactive dialog box																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, marker files have an MRK extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

MarkerId

Specifies the identifier of the marker to save.

ControlFlag

Reserved for future expansion, and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasSetMarker

Synopsis

Set an essential characteristic of a marker.

Syntax

```
void MmeasSetMarker(
    AIL_ID MarkerId,               //out
    AIL_INT64 CharacteristicToSet, //in
    AIL_DOUBLE FirstValue,         //in
    AIL_DOUBLE SecondValue         //in
)
```

Description

This function sets an essential characteristic of a marker. Essential characteristics are used in determining the location of an edge, stripe, or circle marker in a target image, or specifying the position of point markers, which are generally used as a reference marker. MmeasFindMarker() locates the edges, stripes, or circle that best correspond to the specified marker's characteristics. A marker's characteristics should describe the marker as accurately as possible to ensure that it will be successfully located in an image.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MmeasSetMarker()** internally calls the *AIL_DOUBLE* version of this function (MmeasSetMarkerDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MmeasSetMarkerInt64()).

Parameters

MarkerId

Specifies the identifier of the marker.

Markers need to be allocated using MmeasAllocMarker() before being set.

CharacteristicToSet

Specifies the type of characteristic to set.

See the Parameter associations section for possible values that can be specified.

FirstValue

Specifies the first value to assign to the characteristic.

See the Parameter associations section for possible values that can be specified.

SecondValue

Specifies the second value to assign to the characteristic. If only one value is to be returned, it will be returned to FirstValue and SecondValue should be set to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **CharacteristicToSet**, **FirstValue**, and **SecondValue** parameters.

- For M_POINT, M_EDGE, M_STRIPE, and M_CIRCLE markers
- For M_POINT, M_EDGE, and M_STRIPE markers
- For M_EDGE, M_STRIPE, and M_CIRCLE markers

- For M_EDGE and M_STRIPE markers
- For M_POINT markers
- For M_STRIPE markers
- For M_CIRCLE markers

For M_POINT, M_EDGE, M_STRIPE, and M_CIRCLE markers, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_POINT, M_EDGE, M_STRIPE, and M_CIRCLE markers																	
CharacteristicToSet	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstValue																
SecondValue																
M_MARKER_REFERENCE	Sets the offset at which to place the marker's reference position, relative to the marker's actual position. The reference position is only used in calculations between two markers (MmeasCalculate()). • For point markers, Aurora Imaging Library measures the specified offset relative to the X- and Y-coordinates set for the point using M_POSITION. • For edge markers, Aurora Imaging Library measures the specified offset relative to the X- and Y-coordinates of the edge's maximum edgevalue. • For stripe markers, Aurora Imaging Library measures the specified offset relative to the X- and Y-coordinates at the center of a theoretical line between the position (maximum edgevalue) of the stripe's two outermost edges. • For circle markers, Aurora Imaging Library measures the specified offset relative to the X- and Y-coordinates at the circle's center. By default, the marker's reference position is the same as its actual position (the default offset is 0).															
FirstValue	Specifies the X-offset at which to place the marker's reference position. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset value, in the same units as the input coordinate system specified using M_MARKER_REFERENCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

SecondValue	Specifies the Y-offset at which to place the marker's reference position. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset value, in the same units as the input coordinate system specified using M_MARKER_REFERENCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_MARKER_REFERENCE_INPUT_UNITS	Sets the units with which to interpret M_MARKER_REFERENCE. This essentially sets the input coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. If world units are specified, calling MmeasDraw() to draw expected marker characteristics or MmeasFindMarker() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

		M_WORLD	Specifies that results are returned in world units, according to the relative coordinate system. If world units are specified, calling MmeasGetResult() or MmeasGetResultSingle() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

For M_POINT, M_EDGE, and M_STRIPE markers, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

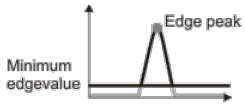
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

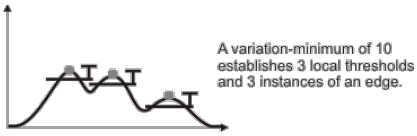
For M_POINT, M_EDGE, and M_STRIPE markers																	
CharacteristicToSet		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstValue																
	SecondValue																
M_NUMBER		Sets the number of edges or stripes to locate in the box search region, or the number of points to explicitly define (points are typically used as a reference marker). In the first case, this can be referred to as the number of marker occurrences. When M_NUMBER is set to a value greater than 1, the marker is considered a multiple-occurrence marker. Note that a circle marker cannot be set as a multiple-occurrence marker. For edges and stripes, unless a minimum number (M_NUMBER_MIN) is specified, no results will be returned if the number of marker occurrences found falls below M_NUMBER.															
	FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that the search should locate all edges or stripes in the box search region. This setting cannot be used for a point marker. If using M_ALL and M_NUMBER_MIN is set to M_DEFAULT, M_NUMBER_MIN is ignored and Aurora Imaging Library will find as many marker occurrences as it can in the image. However, if M_NUMBER_MIN is set to a value other than M_DEFAULT, this value is respected and results are returned, provided that the specified minimum number of markers are found. > Note that when working with a very large image with many stripes, searching for an unknown number of stripes can significantly increase processing time.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of edges or stripes to locate, or point markers to define.	c		e	f	g	a		i		j	k	l	m	n	b

For M_EDGE, M_STRIPE, and M_CIRCLE markers, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_EDGE, M_STRIPE, and M_CIRCLE markers																	
CharacteristicToSet	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstValue																	
SecondValue																	
M_EDGEVALUE_MIN	Sets the minimum edgevalue required for it to be considered an occurrence of the marker.  Single peak that satisfies the minimum edgevalue criterion Aurora Imaging Library establishes edgevalues by taking the normalized first derivative of the intensity profile, which generally resembles a peak when an edge is present. When searching for multiple edges, you might need to further constrain the edgevalues that Aurora Imaging Library considers, using M_EDGEVALUE_VAR_MIN.															
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum edgevalue. A setting of 0.0 indicates that Aurora Imaging Library considers all edgevalues, while a setting of 100.0 indicates that Aurora Imaging Library only considers the greatest possible edgevalue. The greatest possible edgevalue results from a perfectly straight, perfectly white to perfectly black (or vice versa) grayscale transition over one pixel. If you set the minimum edgevalue to a very low value, MmeasFindMarker() can take longer to execute.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEVALUE_VAR_MIN	Sets the minimum prominence required for an edge peak, for it to be considered an occurrence of the marker. To determine if the minimum prominence is met, Aurora Imaging Library establishes a local threshold for each peak by subtracting M_EDGEVALUE_VAR_MIN from it.  The portion of the profile above the local threshold that contains the edge peak can be referred to as the peak's interval, provided that the profile eventually dips below the local threshold on both sides of its edge peak. For Aurora Imaging Library to consider a peak to be an edge, there must be no edgevalues greater than that peak within its interval. If the local threshold calculated with M_EDGEVALUE_VAR_MIN is less than M_EDGEVALUE_MIN, the local threshold equals M_EDGEVALUE_MIN. M_EDGEVALUE_VAR_MIN is typically used by advanced applications where you are trying to find multiple neighboring edges in the search region, which are formed such that the foreground of the first edge becomes the background of the next edge.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum prominence. A setting of 0.0 indicates no prominence; that is, Aurora Imaging Library only considers the maximum edgevalue of the peak's interval. A setting of 100.0 (default) indicates the greatest possible prominence; that is, Aurora Imaging Library considers all edgevalues of the peak's interval that are above M_EDGEVALUE_MIN. In this case, M_EDGEVALUE_VAR_MIN has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS	Sets the degree of smoothness (strength of denoising) applied to the internal projection buffer of the search region during the edge extraction. The recursive edge extraction process involves a denoising operation. Note that M_FILTER_SMOOTHNESS is only available if M_FILTER_TYPE is set to M_SHEN.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 <= *Value* <= 100.0	Specifies the smoothness value. A value of 100.0 results in a strong noise reduction effect, while a value of 0.0 has almost no noise reduction effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE			Sets the type of the first derivative filter with which to perform the edge extraction. The edgevalue calculated for each projection value of the search region is dependent on this setting.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_EULER.	c		e	f	g	a		i		j	k	l	m	n	b
		M_EULER	Specifies an Euler FIR filter. This filter performs a non-recursive edge extraction operation with a predefined kernel of size 2: [-1,1]. This filter is faster but more sensitive to noise, compared to M_PREWITT filter.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PREWITT	Specifies a Prewitt FIR filter. This filter performs a non-recursive edge extraction operation with a predefined kernel of size 3: [-1,0,1]. This filter is slower but less sensitive to noise, compared to M_EULER filter.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SHEN	Specifies a Shen-Castan Infinite Support Exponential filter. This is an exponential weighting function, of the general form: $Ke^{-\beta\lvert n\rvert}$ This filter performs a recursive edge extraction operation using an IIR filter. If this filter actually used a kernel, the kernel size would be theoretically infinite. This IIR filter is slower than FIR filters (M_EULER or M_PREWITT). However, this filter is less sensitive to noise and provides more accurate results. You can also control the strength of denoising applied to the projection buffer of the search region during the edge extraction, using the M_FILTER_SMOOTHNESS setting.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX_ASSOCIATION_DISTANCE	Sets the maximum distance between a marker's edge (either straight or circular) and its associated subedges. The subedges' positions are points extracted from the marker's subregions that are used during the fit operation. For edge and stripe markers, M_MAX_ASSOCIATION_DISTANCE is measured perpendicular from the fitted edge position, along to the search direction. For circle markers, M_MAX_ASSOCIATION_DISTANCE is measured radially from the fitted circle perimeter. Subedges falling outside the specified maximum distance are considered outliers and are not used to perform the marker's line or circle fit operation. For edge and stripe markers, this leads to fitted edges more in line with the marker orientation and, for circle markers, this ensures that the fitted circle will be fitted on subedges roughly at the same distance from the fitted circle center.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MAX_POSSIBLE_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_POSSIBLE_VALUE	Specifies that there is no maximum distance; the entire search region is searched for subedges.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the distance, in the units of the input coordinate system specified using M_MAX_ASSOCIATION_DISTANCE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_ASSOCIATION_DISTANCE_INPUT_UNITS	Sets the units with which to interpret M_MAX_ASSOCIATION_DISTANCE. This essentially sets the input coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. If world units are specified, calling MmeasDraw() to draw expected marker characteristics or MmeasFindMarker() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

M_POLARITY		Specifies the polarity of the marker. Polarity indicates whether an edge must be a rising edge (increase in grayscale value) or a falling edge (decrease in grayscale value).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue	Specifies the polarity of the edge. For stripe markers, this is the polarity of the first edge. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that polarity is not considered.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NEGATIVE	Specifies that the polarity of the edge is negative (falling edge). For circle markers, this indicates that the inside of the ring-shaped region has a greater grayscale value (falling edge from inside to outside).	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITIVE	Specifies that the polarity of the edge is positive (rising edge). For circle markers, this indicates that the inside of the ring-shaped region has a lower grayscale value (rising edge from inside to outside).	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue	Specifies the polarity of the second edge of a stripe. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_OPPOSITE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies that this parameter does not apply. This setting must be used for edge and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the polarity is not considered.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NEGATIVE	Specifies that the polarity of the second edge is negative (falling edge).	c		e	f	g	a		i		j	k	l	m	n	b
		M_OPPOSITE	Specifies that the polarity of the second edge of a stripe marker is the opposite of that of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITIVE	Specifies that the polarity of the second edge is positive (rising edge).	c		e	f	g	a		i		j	k	l	m	n	b
		M_SAME	Specifies that the polarity of the second edge of a stripe marker is the same as that of the first edge.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_REGION_CLIPPING	Sets whether Aurora Imaging Library can automatically clip the marker's search region, when it falls outside the image. This setting is ignored unless the search region's size is explicitly set. That is, the values specified with M_BOX_SIZE are not set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the search region cannot be automatically clipped. In this case, if the search region falls outside the image, the marker cannot be found (unless you are using subregions).	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMIZE_AREA	Specifies that, if the search region falls outside the image, Aurora Imaging Library can automatically clip the region according to the best valid area in which to search, and the marker can still be found. The modified region is internally created by Aurora Imaging Library; the actual search region that you defined is never altered. The automatically determined region must be inscribed by the defined search region, must be within the boundaries of the image, and must meet the specified M_SEARCH_REGION_CLIPPING_... conditions. Aurora Imaging Library will not clip the region if it cannot adhere to all of these requirements.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_MIN_AREA	Sets the minimum area requirement for the clipped region that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum area required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_MIN_HEIGHT	Sets the minimum width requirement for the clipped region that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the minimum height required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_MIN_WIDTH			Sets the minimum width requirement for the clipped region that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0.0	Specifies the minimum width required for the clipped region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_CLIPPING_PRESERVE_CENTER			Sets the center requirement for the clipped region that can be internally created when using M_SEARCH_REGION_CLIPPING with M_MAXIMIZE_AREA. All the settings are available to edge and stripe markers, but only M_DEFAULT and M_AUTO are available to circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ALONG_HEIGHT	Specifies that the center, along the height of the defined search region, must be preserved in the clipped region. This setting is not available for circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ALONG_WIDTH	Specifies that the center, along the width of the defined search region, must be preserved in the clipped region. This setting is not available for circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
		M_AUTO	Specifies that Aurora Imaging Library will determine how the center of the defined search region will be preserved in the clipped region. For edge and stripe markers, M_AUTO is the same as M_NONE. For circle markers, M_AUTO is the only possible value.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NONE	Specifies that the clipped region need not preserve the center of the defined search region. This setting is not available for circle markers.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_REGION_INPUT_UNITS	Sets the units with which to interpret M_BOX_CENTER, M_BOX_ORIGIN, M_BOX_SIZE, M_BOX_ANGLE, M_BOX_ANGLE_ACCURACY, M_BOX_ANGLE_TOLERANCE, M_BOX_ANGLE_DELTA_POS, M_BOX_ANGLE_DELTA_NEG, M_RING_CENTER, M_RING_RADII, and M_SUB_REGIONS_CHORD_ANGLE. This essentially sets the input coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units, according to the relative coordinate system. If world units are specified, calling MmeasDraw() to draw expected marker characteristics or MmeasFindMarker() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_NUMBER	Sets the number of sections in which to divide the search region. Since a subregion of each section is processed, this number also corresponds to the number of subregions to process. For circle markers, this value will give each subregion the same area; that is, each subregion will occupy (360/M_SUB_REGIONS_NUMBER) ^{o} of the circle. For edge and stripe markers, if you specify one subregion when multiple subregions are required (for example, when using MmeasFindMarker() with M_DEFAULT, M_ANGLE, or M_LINE_EQUATION), Aurora Imaging Library internally uses three subregions.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. For edge and stripe markers, the default is one. For circle markers, the default is eight.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* < 8	Specifies a number of sections (subregions) that can only be used for edge and stripe markers.	c		e	f	g	a		i		j	k	l	m	n	b

		Value >= 8	Specifies a number of sections (subregions) that can be used for edge, stripe, and circle markers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBPIXEL_MODE			Sets how Aurora Imaging Library calculates subpixel positions.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_LOCAL.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GLOBAL	Specifies that Aurora Imaging Library calculates subpixel positions based on the entire region within which the edge is found. This mode is recommended when measuring wide edges.	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOCAL	Specifies that Aurora Imaging Library calculates subpixel positions based on the immediate neighborhood within which the edge's maximum edgevalue is found. This mode offers more precision, but can be less stable with wide edges.	c		e	f	g	a		i		j	k	l	m	n	b

For M_EDGE and M_STRIPE markers, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

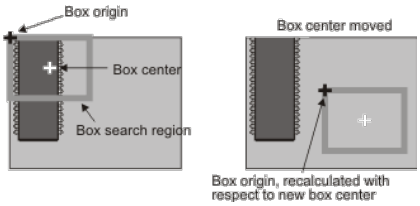
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

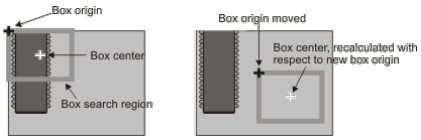
For M_EDGE and M_STRIPE markers																		
CharacteristicToSet			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstValue																	
	SecondValue																	
M_BOX_ANGLE			Sets the angle of the box search region. The center of rotation of the box search region is set with M_BOX_ANGLE_REFERENCE.															
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 0°.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the content of the box search region is analyzed to automatically determine the angle of the marker.	c		e	f	g	a		i		j	k	l	m	n	b

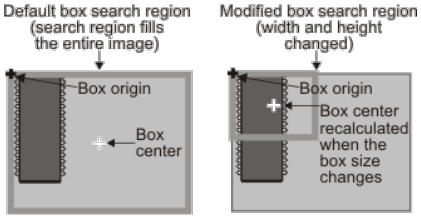
		0 <= *Value* <= 360	Specifies the angle of the box search region, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. An angle interpreted using the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle according to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_ACCURACY			Sets the accuracy of the angular search. This determines the size of the step angle to use once the approximate location of the marker is found.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DISABLE	Specifies the angle of accuracy to be equal to the angle of tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
		0.1 <= *Value* <= 180.0	Specifies the accuracy of the angular search, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_DELTA_NEG			Sets the negative range of angles within which to search for the marker. The search is performed between the range of angles defined by: (M_BOX_ANGLE - M_BOX_ANGLE_DELTA_NEG) to (M_BOX_ANGLE + M_BOX_ANGLE_DELTA_POS), inclusively, starting with an angle closest to that of M_BOX_ANGLE. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b

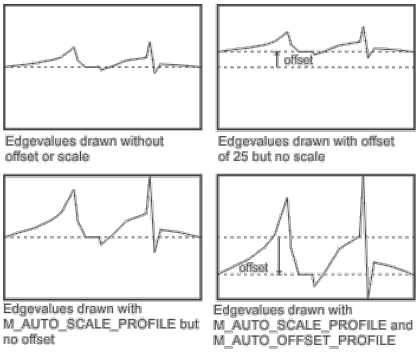
		M_DEFAULT	Specifies the default negative range of angles, in degrees. For a symmetrical stripe marker (a marker set with M_OPPOSITE), the default is 180.0. For an edge marker or a non-symmetrical stripe marker, the default is 360.0.	c		e	f	g	a		i		j	k	l	m	n	b
		0.1 <= *Value* <= 360.0	Specifies the negative range of angles, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_DELTA_POS			Sets the positive range of angles within which to search for the marker. The search is performed between the range of angles defined by: (M_BOX_ANGLE - M_BOX_ANGLE_DELTA_NEG) to (M_BOX_ANGLE + M_BOX_ANGLE_DELTA_POS), inclusively, starting with an angle closest to that of M_BOX_ANGLE. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default positive range of angles, in degrees. For a symmetrical stripe marker (a marker set with M_OPPOSITE), the default is 180.0. For an edge marker or a non-symmetrical stripe marker, the default is 360.0.	c		e	f	g	a		i		j	k	l	m	n	b
		0.1 <= *Value* <= 360.0	Specifies the positive range of angles, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_MODE			Enables the use of multiple-angle search, as specified by the settings of other M_BOX_ANGLE... characteristics. The angle of the marker with the highest match score is returned. For an edge marker, this is the score of the edge, and for a stripe marker, this is the mean score of the edges.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DISABLE	Specifies that the search should only be performed at the angle specified by M_BOX_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ENABLE	Specifies that multiple-angle search is enabled.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_REFERENCE			Sets the center of rotation to use when M_BOX_ANGLE is not zero. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_BOX_CENTER.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BOX_CENTER	Specifies the center of the box search region as the center of rotation. Use the M_BOX_CENTER characteristic to set the coordinates of the center of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BOX_ORIGIN	Specifies the top-left corner of the box search region as the center of rotation. Use the M_BOX_ORIGIN characteristic to set the coordinates of the top-left corner of the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ANGLE_TOLERANCE			Sets the rotation tolerance of the marker. This is the full range of degrees within which a marker can be rotated from a box search region that is at a specific angle and still be found. This determines the step angle used for a multiple-angle search.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 5.0.	c		e	f	g	a		i		j	k	l	m	n	b
		0.1 <= *Value* <= 360.0	Specifies the rotation tolerance, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

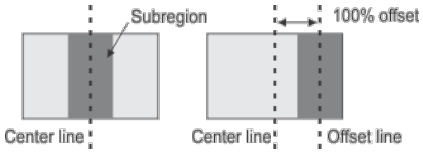
M_BOX_CENTER		Sets the coordinates of the center of the box search region. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE). When you change M_BOX_CENTER, Aurora Imaging Library recalculates M_BOX_ORIGIN, relative to the new center position. 	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue	Specifies the X-coordinate of the center of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default X-coordinate of the center of the box search region. This depends on the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. If using pixel units, the default center is the center of the pixel coordinate system. If using world units, the default center is the origin of the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the center of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue	Specifies the Y-coordinate of the center of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default Y-coordinate of the center of the box search region. This depends on the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. If using pixel units, the default center is the center of the pixel coordinate system. If using world units, the default center is the origin of the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate of the center of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOX_ORIGIN		Sets the coordinates of the top-left corner of the box search region. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE). When you change M_BOX_ORIGIN, Aurora Imaging Library recalculates M_BOX_CENTER, relative to the new box origin. 	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue	Specifies the X-coordinate of the origin of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT Specifies the default X-coordinate of the origin of the box search region. The default origin is the origin of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
		Value Specifies the X-coordinate of the origin of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue	Specifies the Y-coordinate of the origin of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT Specifies the default Y-coordinate of the origin of the box search region. The default origin is the origin of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
		Value Specifies the Y-coordinate of the origin of the box search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOX_SIZE		Sets the width and height of the box search region. If M_SEARCH_REGION_INPUT_UNITS is set to M_WORLD, the Measurement module uses the world dimensions specified to estimate a rectangular box in pixel units. The module will then use this approximation for internal calculations. Note that the search region must fit entirely inside the target image, unless subregions are used or clipping is enabled. If don't use subregions or enable clipping, and you choose a location, size, or search angle that causes the box to extend beyond the edge of the target image, the marker cannot be found (MmeasGetResult() with M_VALID_FLAG will return M_FALSE). When you set the width and height of the search region, the search region expands or shrinks with respect to its origin, set with M_BOX_ORIGIN. When the width or height changes, Aurora Imaging Library recalculates M_BOX_CENTER, relative to the box search region's origin.  Note that if you have enabled clipping, the size of the box search region internally use might be smaller than the specified size. This is only a temporary effect; it does not change the specified size. For more information, see the Search region section of Chapter 19: Measurement.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue	Specifies the width of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies to use the width of the entire target image. If you specify M_DEFAULT, and M_SEARCH_REGION_INPUT_UNITS is set to M_WORLD, you must also set the SecondValue parameter to M_DEFAULT; otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the width of the box search region within the target image, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. This value must be positive.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue	Specifies the height of the box search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies to use the height of the entire target image. If you specify M_DEFAULT, and M_SEARCH_REGION_INPUT_UNITS is set to M_WORLD, you must also set the FirstValue parameter to M_DEFAULT; otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the height of the box search region within the target image, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. This value must be positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PROFILE_SCALE_OFFSET			Sets the scale and offset to use when drawing the edge profile or edgevalue characteristics using MmeasDraw() with M_DRAW_EDGES_PROFILE, M_DRAW_EDGEVALUE_MIN_IN_PROFILE, M_DRAW_EDGEVALUE_PEAK_WIDTH_IN_PROFILE, or M_DRAW_POSITION_IN_PROFILE.  Edgevalues drawn without offset or scale Edgevalues drawn with offset of 25 but no scale Edgevalues drawn with M_AUTO_SCALE_PROFILE but no offset Edgevalues drawn with M_AUTO_SCALE_PROFILE and M_AUTO_OFFSET_PROFILE	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Sets the scale to use when drawing. The shape and proportions of the drawing are kept the same; only the size is affected. This can be useful for applications where the information being drawn is too small or too large to see clearly. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_AUTO_SCALE_PROFILE	Specifies to automatically compute the greatest scale factor that will fit the entire drawing within the destination buffer (or within the area that corresponds to the search region, if M_DRAW_IN_BOX is combined when drawing) after the offset has been applied. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value > 0.0	Specifies the scale multiplier.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue		Sets the offset at which to draw the information from the middle of the height of the destination buffer or the area corresponding to the search region. The shape and proportions of the drawing are kept the same; only the location is affected. The middle is described as the center height of the image buffer or the profile's associated search region, and is at an offset of 0. INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_AUTO_OFFSET_PROFILE	Specifies to automatically compute the offset such that the maximum and minimum of the drawing are touching the ends of the destination buffer (or the area that corresponds to the search region, if M_DRAW_IN_BOX is combined when drawing). This value can only be used when FirstValue is set to M_AUTO_SCALE_PROFILE. Note that this value is optimized for M_DRAW_EDGES_PROFILE. Using M_AUTO_OFFSET_PROFILE with other M_DRAW_..._PROFILE values might draw outside of the destination (image buffer or the area corresponding to the search region). This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the offset from the middle of the height of the destination buffer or the area corresponding to the search region. The ends of the destination buffer or the area corresponding to the search region correspond to offsets -100 and 100. There is no limit to the offset value. However, large values might cause the required drawing to be drawn outside the visible area.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MIN			Sets the minimum number of edges or stripes to locate in the box search region.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value. The default number of edges or stripes is the same as the value set in M_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the minimum number of edges or stripes to locate.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIENTATION			Sets the orientation of the edge or stripe marker.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_VERTICAL.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the orientation of the marker is unknown; both horizontal and vertical orientations are searched.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HORIZONTAL	Specifies that the marker has a horizontal orientation.	c		e	f	g	a		i		j	k	l	m	n	b

		M_VERTICAL	Specifies that the marker has a vertical orientation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_REGION_ANGLE_INTERPOLATION_MODE			Sets the type of interpolation to use when searching for markers. This only has an effect when either M_BOX_ANGLE_MODE is enabled or M_BOX_ANGLE is set to a value other than 0.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_BILINEAR.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_OFFSET			Sets the offset of the subregions, from the center point of each of their sections. Specify the offset as a percentage of the distance from the center point of the section to the maximum possible offset in each section. 100 corresponds to the maximum possible offset. The direction of the offset (X or Y) is taken from M_ORIENTATION, with positive values shifting the subregion down or to the right, and negative values shifting up or to the left.  Note that subregions are offset as a group.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
		-100 <= *Value* <= 100	Specifies the offset of the subregions.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUB_REGIONS_SIZE	Sets the size of the subregions, as a percentage of their section. Subregions are always rectangular and extend from one edge of the box search region to the other in the orientation direction. If made smaller than their section, the subregions become thinner and occupy the specified percentage of their section. In this case, subregions will remain centered unless moved with M_SUB_REGIONS_OFFSET. Note that the size of subregions are set as a group.	c		e	f	g	a		i		j	k	l	m	n	b
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100%.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the size of the subregions, as a percentage. If the size is set to 100, subregions will be evenly distributed within the box search region, with no space between them.	c		e	f	g	a		i		j	k	l	m	n	b

For M_POINT markers only, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_POINT markers																
CharacteristicToSet	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
FirstValue																
SecondValue																
M_MULTIPLE_POINT_ANGLE	Sets the angle at which subsequent points are placed, in a theoretical line, for a multiple point marker (at the interval set with M_SPACING and proceeding from the coordinates defined by M_POSITION).															
FirstValue	Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0°.	c		e	f	g	a		i		j	k	l	m	n	b

		0 <= *Value* <= 360	Specifies the angle for a multiple point marker, in degrees, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS. An angle interpreted according to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle according to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_INPUT_UNITS			Sets the units with which to interpret M_POSITION, M_SPACING, and M_MULTIPLE_POINT_ANGLE. This essentially sets the input coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PIXEL	Specifies to interpret the values in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		M_WORLD	Specifies to interpret the values in world units, according to the relative coordinate system. If world units are specified, calling MmeasDraw() to draw expected marker characteristics generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION			Sets the coordinates of a point marker. For a multiple point marker, this defines the position of the first point.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies the X-coordinate of the point marker. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the position is invalid. When a point marker is allocated, M_POSITION is set to M_ANY. After allocation, you must explicitly set a position (X-coordinate, Y-coordinate); otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the point marker, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue		Specifies the Y-coordinate of the point marker. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b

		M_ANY	Specifies that the position is invalid. When a point marker is allocated, M_POSITION is set to M_ANY. After allocation, you must explicitly set a position (X-coordinate, Y-coordinate); otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the point marker, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING			Sets the spacing (distance) between points of a point marker. This setting is only available for multiple point markers.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that the spacing is invalid. When a point marker is allocated, M_SPACING is set to M_ANY. After allocation, you must explicitly set the spacing; otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the spacing, in the units of the input coordinate system specified using M_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

For M_STRIPE markers only, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_STRIPE markers																		
CharacteristicToSet			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstValue																	
	SecondValue																	
M_INCLUSION_POINT			Sets a position (inclusion point) in the target image that must either be included or not included in the stripe marker found, depending on M_INCLUSION_POINT_INSIDE_STRIPE. If you specify an inclusion point, you must also set M_INCLUSION_POINT_INSIDE_STRIPE to M_YES or M_NO; otherwise, the specified inclusion point is ignored. INQ															
	FirstValue		Specifies the X-coordinate of the inclusion point.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b

		M_ANY	Specifies no inclusion point. In this case, M_INCLUSION_POINT_INSIDE_STRIPE is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the inclusion point's X-coordinate, in the units of the input coordinate system specified using M_INCLUSION_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue		Specifies the Y-coordinate of the inclusion point.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies no inclusion point. In this case, M_INCLUSION_POINT_INSIDE_STRIPE is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the inclusion point's Y-coordinate, in the units of the input coordinate system specified using M_INCLUSION_POINT_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUSION_POINT_INPUT_UNITS			Sets the units with which to interpret M_INCLUSION_POINT. This essentially sets the input coordinate system to use.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PIXEL	Specifies to interpret the value in pixel units, according to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		M_WORLD	Specifies to interpret the value in world units, according to the relative coordinate system. If world units are specified, calling MmeasDraw() to draw expected marker characteristics or MmeasFindMarker() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_INCLUSION_POINT_INSIDE_STRIPE			Sets whether the stripe marker found must contain the inclusion point (M_INCLUSION_POINT).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_ANY.	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANY	Specifies that M_INCLUSION_POINT is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

		M_NO	Specifies that only stripes that have the inclusion point falling outside its two outer edges can be found.	c		e	f	g	a		i		j	k	l	m	n	b
		M_YES	Specifies that only stripes that have the inclusion point falling between its two outer edges can be found.	c		e	f	g	a		i		j	k	l	m	n	b

For M_CIRCLE markers only, the CharacteristicToSet and corresponding FirstValue and SecondValue parameters can be set to the following.

Note that the SecondValue parameter should be set to **M_NULL** if no parameter setting is required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_CIRCLE markers																		
CharacteristicToSet			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	FirstValue																	
	SecondValue																	
M_CIRCLE_ACCURACY			Sets the accuracy of the circle's fit operation.															
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HIGH	Specifies that the circle's fit operation will be performed with high accuracy. This typically takes longer than low accuracy, but can produce more precise results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOW	Specifies that the circle's fit operation will be performed with low accuracy. This is typically faster than high accuracy, but can produce less precise results, particularly with noisy or badly contrasted images.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE_INSIDE_SEARCH_REGION			Sets whether the circle's fit operation can result in a circle that is outside the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DISABLE	Specifies that the circle's fit operation can result in a circle that is outside the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b

		M_ENABLE	Specifies that the circle's fit operation cannot result in a circle that is outside the ring search region.	c		e	f	g	a		i		j	k	l	m	n	b
M_RING_CENTER			Sets the coordinates of the center of the ring search region. The ring's center is considered to be its position. Note that at least three valid subregions must be within the target image for the circle marker to be found (even if clipping is enabled).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies the X-coordinate of the center of the ring search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default X-coordinate of the center of the ring search region. This depends on the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. If using pixel units, the default center is the center of the pixel coordinate system. If using world units, the default center is the origin of the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the X-coordinate of the center of the ring search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue		Specifies the Y-coordinate of the center of the ring search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default Y-coordinate of the center of the ring search region. This depends on the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. If using pixel units, the default center is the center of the pixel coordinate system. If using world units, the default center is the origin of the relative coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the Y-coordinate of the center of the ring search region, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_RING_RADII			Sets the inner and outer radius of the ring search region. The radii values are set according to the region's center (M_RING_CENTER). The inner and outer radius must be set concurrently. The inner radius must be less than the outer radius; this creates a ring within which the circle must be found.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies the inner radius of the ring search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies an inner radius of 0.0. If the outer radius is set to M_DEFAULT, the inner radius must also be set to M_DEFAULT; otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0	Specifies the inner radius, relative to the center of the ring search region, in the units specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
	SecondValue		Specifies the outer radius of the ring search region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default outer radius. This value corresponds to either half the target image's width (X-size x 0.5) or half its height (Y-size x 0.5), depending on which is smaller. Note that if the outer radius is set to M_DEFAULT, the inner radius must also be set to M_DEFAULT; otherwise, an error will occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Value > *Inner radius*	Specifies the outer radius, relative to the center of the ring search region, in the units specified using M_SEARCH_REGION_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_REGIONS_CHORD_ANGLE			Sets the angle with which to set the ring search region's outer ring segment (chord). The projection of this chord segment determines the dimension of the rectangular subregion that is perpendicular to radial direction.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstValue		Specifies one of the following.INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 10.0 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
		0.0 < *Value* <= 45.0	Specifies the angle, in degrees, in the units of the input coordinate system specified using M_SEARCH_REGION_INPUT_UNITS. An angle interpreted according to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle according to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MmeasSetMarkerInt64 (AIL_ID MarkerId, AIL_INT64 CharacteristicToSet, AIL_INT64 FirstValue, AIL_INT64 SecondValue)
```

Details

Parameters

MarkerId

See MarkerId of the main function for a description.

CharacteristicToSet

See CharacteristicToSet of the main function for a description.

FirstValue

See FirstValue of the main function for a description.

SecondValue

See SecondValue of the main function for a description.

```
void MmeasSetMarkerDouble (AIL_ID MarkerId, AIL_INT64 CharacteristicToSet, AIL_DOUBLE FirstValue, AIL_DOUBLE SecondValue)
```

Details

Parameters

MarkerId

See MarkerId of the main function for a description.

CharacteristicToSet

See CharacteristicToSet of the main function for a description.

FirstValue

See FirstValue of the main function for a description.

SecondValue

See SecondValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasSetScore

Synopsis

Set a score characteristic of a marker.

Syntax

```
void MmeasSetScore(
    AIL_ID MarkerId,          //out
    AIL_INT64 Characteristic, //in
    AIL_DOUBLE Min,           //in
    AIL_DOUBLE Low,           //in
    AIL_DOUBLE High,          //in
    AIL_DOUBLE Max,           //in
    AIL_DOUBLE ScoreOffset,   //in
    AIL_INT64 InputUnits,     //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function sets a score characteristic of a marker. Aurora Imaging Library uses score characteristics to determine the best edge, stripe, or circle marker. This function only processes markers that have all the essential characteristics specified with MmeasSetMarker().

Unlike essential characteristics, which are either met or not, score characteristics can have a range of acceptable values; based on where the actual characteristic result falls within these specified limits, Aurora Imaging Library calculates the characteristic's score. To calculate the final score of the marker itself (MmeasGetResult() with M_SCORE_TOTAL), Aurora Imaging Library multiplies the score of all the marker's score characteristics. The marker with the highest final score is the one located by MmeasFindMarker().

To retrieve the characteristic's score, as well as other related information, such as the actual value of the characteristic (for example, the contrast of the marker in the image), you can use MmeasGetScore() with M_RESULT (after calling MmeasFindMarker()). To inquire the specified settings for a score characteristic, use MmeasGetScore() with M_MARKER.

To set the acceptable limits of a score characteristic, use the Min, Low, High, and Max parameters. The values for these parameters must adhere to the following conditions: Min <= Low <= High <= Max . If these conditions are not met, Aurora Imaging Library generates an error. Characteristics that are greater than or equal to Low and less than or equal to High receive a score of 100%, while those that are greater than or equal to Min and less than Low, or greater than High and less than or equal to Max, receive a score above 0% and below 100%. Characteristics that are less than Min or greater than Max receive a score of 0% and are discarded.

You can set all limits except Min to M_MAX_POSSIBLE_VALUE, which represents a predetermined maximum established by Aurora Imaging Library for that characteristic (the smallest possible value for any characteristic is always 0). Once a limit has been set to M_MAX_POSSIBLE_VALUE, all subsequent limits must also be set to M_MAX_POSSIBLE_VALUE. For example, if you are using M_EDGE_CONTRAST_SCORE, and set the Low parameter to M_MAX_POSSIBLE_VALUE, you must also set the High and Max parameters to M_MAX_POSSIBLE_VALUE.

By default, Aurora Imaging Library only calculates the score of the marker's edge strength (M_STRENGTH_SCORE); therefore, the marker with the highest edgevalue (and all the essential characteristics) is found. When using another score characteristic to find the best marker, you might want to ignore the strength characteristic, to avoid its multiplicative influence. To do so, specify M_STRENGTH_SCORE and set Min and Low to 0, and set High and Max to M_MAX_POSSIBLE_VALUE.

You can reduce the influence of a characteristic on the total score by using a score offset. Aurora Imaging Library calculates the lowest-possible nonzero score by subtracting the specified score-offset from 100% (100% - ScoreOffset

= *LowestPossibleScore*). Since characteristics can, by default, score above 0%, the default score-offset is equivalent to 100% (calculated as, 100% - 100% = 0%). Unless you are developing an advanced application, the default is typically appropriate. Values below Min and above Max are set to 0, whereas values between Min and Low are mapped to scores between this *LowestPossibleScore* and 100%, and values between High and Max are mapped to scores between 100% and this *LowestPossibleScore*.

Parameters

MarkerId

Specifies the identifier of the measurement marker for which to set the score characteristic.

Characteristic

Specifies the score characteristic to set.

The values in the table below are for specifying a score characteristic for an edge, stripe, or circle marker.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a score characteristic for an edge, stripe, or circle marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_CONTRAST_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the grayscale difference between the start and end of the intensity transition from which the marker's edge is established. To retrieve the start and end of the intensity transition, use MmeasGetResult() with M_EDGE_START and M_EDGE_END. The theoretical limit of the greatest possible contrast value (M_MAX_POSSIBLE_VALUE) depends on the target image. For example, for an 8-bit image, the greatest possible contrast is 255.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEVALUE_PEAK_CONTRAST_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the grayscale difference of the intensity transition between the first zero edgevalues on both sides of the established edge peak. To retrieve the minimum and maximum position of the edge peak, use MmeasGetResult() with M_EDGEVALUE_PEAK_POS_MIN and M_EDGEVALUE_PEAK_POS_MAX. Note that some search regions might never have a zero edgevalue; in which case, you should use M_EDGE_CONTRAST_SCORE for a more meaningful result. The theoretical limit of the greatest possible contrast value (M_MAX_POSSIBLE_VALUE) depends on the target image. For example, for an 8-bit image, the greatest possible contrast is 255; this corresponds to an edgevalue of 100.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRENGTH_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the mean edgevalue (strength) of the marker. The theoretical limit of the greatest possible edgevalue (M_MAX_POSSIBLE_VALUE) is 100.	c		e	f	g	a		i		j	k	l	m	n	b

The values in the table below are for specifying a score characteristic for an edge or stripe marker.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a score characteristic for an edge or stripe marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISTANCE_FROM_BOX_ORIGIN_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the distance between the position of the box search region's origin and the position of the marker. The theoretical limit of the greatest possible position-offset (M_MAX_POSSIBLE_VALUE) depends on the relevant dimension of the box search region (either the box's width or height, depending on the orientation). For a multiple-occurrence marker, each occurrence must respect the specified limits; otherwise Aurora Imaging Library discards the marker.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPACING_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the typical distance (spacing) between consecutive edge or stripe occurrences of a multiple-occurrence marker. The theoretical limit of the greatest possible spacing value (M_MAX_POSSIBLE_VALUE) depends on the relevant dimension of the box search region (either the box's width or height, depending on the orientation).	c		e	f	g	a		i		j	k	l	m	n	b

The values in the table below are for specifying a score characteristic for a stripe marker.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a score characteristic for a stripe marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_INSIDE_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the number of edges inside a stripe marker. The theoretical limit of the greatest possible number of edges (M_MAX_POSSIBLE_VALUE) is equivalent to half the relevant dimension of the box search region (either 0.5 x *BoxWidth* or 0.5 x *BoxHeight*, depending on the orientation).	c		e	f	g	a		i		j	k	l	m	n	b

M_STRIPE_WIDTH_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the distance (width) between the stripe's two outermost edges. The theoretical limit of the greatest possible width value for the stripe (M_MAX_POSSIBLE_VALUE) depends on the relevant dimension of the box search region (either the box's width or height, depending on the orientation).	c		e	f	g	a		i		j	k	l	m	n	b

The values in the table below are for specifying a score characteristic for a circle marker.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a score characteristic for a circle marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RADIUS_SCORE	Specifies that Aurora Imaging Library must calculate a score, as a percentage, based on the linear distance between the circle's center and its circumference (radius). The theoretical limit of the greatest possible radius value for the circle (M_MAX_POSSIBLE_VALUE) is the length of the outer radius specified for the ring search region using MmeasSetMarker() with M_RING_RADII.	c		e	f	g	a		i		j	k	l	m	n	b

Min

Specifies the minimum limit, which Aurora Imaging Library uses to calculate the characteristic's score. Any value less than the minimum limit receives a score of 0%.

For M_STRENGTH_SCORE, this value is set to 0 by default.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum acceptable limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Value* < *Low*	Specifies the minimum acceptable limit. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Low

Specifies the low limit, which Aurora Imaging Library uses to calculate the characteristic's score. Any value less than the lowest limit and greater than or equal to the minimum limit receives a score above 0% and below 100%, while any value greater than or equal to the lowest limit and less than or equal to the highest limit receives a score of 100%.

For M_STRENGTH_SCORE, this value is set to M_MAX_POSSIBLE_VALUE by default.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the low acceptable limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_POSSIBLE_VALUE	Specifies that the low acceptable limit is equivalent to the greatest possible value.	c		e	f	g	a		i		j	k	l	m	n	b

Min <= *Value* <= *High*	Specifies the low acceptable limit. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

High

Specifies the high limit, which Aurora Imaging Library uses to calculate the characteristic's score. Any value greater than the highest limit and less than or equal to the maximum limit receives a score above 0% and below 100%.

For M_STRENGTH_SCORE, this value is set to M_MAX_POSSIBLE_VALUE by default.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the high acceptable limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_POSSIBLE_VALUE	Specifies that the high acceptable limit is equivalent to the greatest possible value. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
Low <= *Value* <= *Max*	Specifies the high acceptable limit.	c		e	f	g	a		i		j	k	l	m	n	b

Max

Specifies the maximum limit, which Aurora Imaging Library uses to calculate the characteristic's score. Any value greater than the maximum limit receives a score of 0%.

For M_STRENGTH_SCORE, this value is set to M_MAX_POSSIBLE_VALUE by default.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum acceptable limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_POSSIBLE_VALUE	Specifies that the maximum acceptable limit is equivalent to the greatest possible value. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
High <= *Value* <= M_MAX_POSSIBLE_VALUE	Specifies the maximum acceptable limit.	c		e	f	g	a		i		j	k	l	m	n	b

ScoreOffset

Specifies the characteristic's score-offset. This allows you to indirectly specify the lowest non-zero score that a characteristic can receive. The measurement module subtracts the score-offset from 100% to establish the lowest possible score assigned to values between Min and above Max and values between Min and Low.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the score-offset																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 100.0%. This means that the lowest possible score is 0 (100%-100%=0).	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies the score-offset, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

InputUnits

Specifies the units with which to interpret the Min, Low, High, and Max parameter values, when applicable. This essentially sets the input coordinate system to use.

You must set this parameter to M_DEFAULT for the following score characteristics: M_EDGE_CONTRAST_SCORE, M_EDGEVALUE_PEAK_CONTRAST_SCORE, M_EDGE_INSIDE_SCORE, and M_STRENGTH_SCORE. In these cases, M_DEFAULT indicates that the input unit information is not required.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the input units																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_PIXEL, if the score characteristic's settings are affected by input units. Otherwise, M_DEFAULT specifies that the input unit information should be ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that the values will be interpreted in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the values will be interpreted in world units, with respect to the relative coordinate system. If world units are specified, calling MmeasFindMarker() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.

MmeasStream

Synopsis

Load, restore, or save a marker from/to a file or a memory stream.

Syntax

```
void MmeasStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *MarkerIdPtr,           //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a marker from/to a file or memory stream.

To inquire the number of bytes necessary to save a marker to memory stream, you should first call this function (**MmeasStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using MmeasSaveMarker().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

All the marker characteristics that were in effect when the marker was saved are restored. This function can also save an allocated marker, including all its characteristics, to a specified file or memory stream.

Using **MmeasStream**(), you can choose to save a backwards-compatible version of the marker object, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore marker objects saved using Aurora Imaging Library version 7.0 with Processing Pack 1 or above. Settings that do not exist in the lower version will be filled with default values when the marker object is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the measurement marker.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the measurement marker.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the measurement marker. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the stream that holds the measurement marker																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the marker object.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

MarkerIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the marker.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the measurement marker, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of a measurement marker will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **MarkerIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																

SysId																
Version																
MarkerIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a marker object to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the marker object must be compatible when saved. The size of a marker object might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
MarkerIdPtr	Specifies the address of the variable containing the identifier of the marker object.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated marker object.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b

		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Marker objects typically have an MRK file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	MarkerIdPtr		Specifies the address of the variable containing the identifier of the marker object. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Restores a marker object from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Marker objects typically have an MRK file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the marker object.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
MarkerIdPtr	Specifies the address of the variable in which to write the identifier of the marker object. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a marker object to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Zebra Aurora software products, when saving a measurement context to a file, use the MRK file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version		Specifies the Aurora Imaging Library version with which the marker object must be compatible when saved. Note that the marker object can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
MarkerIdPtr		Specifies the address of the variable containing the identifier of the marker object to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmeas11.lib.
DLL	Requires ail11.dll; ailmeas11.dll.